

Toward Reference Models for Requirements Traceability

Balasubramaniam Ramesh, *Member, IEEE*, and Matthias Jarke, *Senior Member, IEEE*

Abstract—Requirements traceability is intended to ensure continued alignment between stakeholder requirements and various outputs of the system development process. To be useful, traces must be organized according to some modeling framework. Indeed, several such frameworks have been proposed, mostly based on theoretical considerations or analysis of other literature. This paper, in contrast, follows an empirical approach. Focus groups and interviews conducted in 26 major software development organizations demonstrate a wide range of traceability practices with distinct low-end and high-end users of traceability. From these observations, reference models comprising the most important kinds of traceability links for various development tasks have been synthesized. The resulting models have been validated in case studies and are incorporated in a number of traceability tools. A detailed case study on the use of the models is presented. Four kinds of traceability link types are identified and critical issues that must be resolved for implementing each type and potential solutions are discussed. Implications for the design of next-generation traceability methods and tools are discussed and illustrated.

Index Terms—Requirements traceability, reference models, requirements engineering, traceability models, traceability, traceability framework, traceability practice.

1 INTRODUCTION

REFERENCE models are prototypical models of some application domain, usually organized according to some underlying basic metamodel. The purpose of reference models is to significantly reduce the task of creating application-specific models and systems: The user selects relevant parts of the reference model, adapts them to the problem at hand, and configures an overall solution from these adapted parts. Since the analysis of a domain can take an enormous effort when started from scratch, the use of reference models has been reported to save up to 80 percent in development costs for systems in standardized domains [1]. Not surprisingly, reference models have become highly successful in many industries, the best-known example being the SAP approach.

Not every domain is sufficiently standardized to allow for a reference model of the final product—the system to be built. Moreover, approaches like the one followed by SAP are not necessarily supportive of change, especially when this change goes beyond the initially covered domain. However, at least experience on how to get to the product should be reused [2]. This leads to the idea of reference models for capturing the development process itself, not to be confused with prescriptive software process models [3].

The process of developing reference models is arduous. The traditional normative computer science approach of

imposing such models on developers is long known to have failed in most cases. Reference models are therefore an abstraction of best practice, condensed from numerous case studies over an extended period of time, followed by more case studies to refine and evaluate the proposed reference model. There is nothing provably correct about reference models; they derive their relevance from the slice of practice they cover. Nevertheless, by formalizing a reference model in an appropriate mathematical framework, at least a number of elementary desirable properties can be ensured.

1.1 Why Reference Models for Traceability?

Our interest is the development of reference models for requirements traceability. Summarizing several earlier views and perspectives, Gotel and Finkelstein [4] define requirements traceability as “*the ability to describe and follow the life of a requirement, in both a forward and backward direction, i.e., from its origins, through its development and specification, to its subsequent deployment and use, and through periods of ongoing refinement and iteration in any of these phases.*” The importance of requirements traceability is highlighted by the fact that, for example, the U.S. Department of Defense spends about 4 percent of its IT costs on traceability—often without getting an adequate value for this money as traceability in many organizations is haphazard, the standards provide little guidance, and the models and mechanisms vary to a large degree and are often poorly understood. Not surprisingly, the market for requirements traceability tools is booming even though current tools support only rather simple traceability models and services.

Previous models of requirements traceability focus on different aspects of requirements traceability. Version and configuration management systems focus on the **SOURCE** aspect (i.e., physical artifacts where traceability information is maintained), emphasizing the document management

• B. Ramesh is with the Department of Computer Information Systems, Georgia State University, 35 Broad Street, Atlanta, GA 30303. E-mail: bramesh@gsu.edu.

• M. Jarke is with GMD-FIT, Schloss Birlinghoven, 53754 Sankt Augustin, Germany, and with Informatik V, RWTH Aachen, Ahornstr. 55, 52074 Aachen, Germany. E-mail: jarke@informatik.rwth-aachen.de.

Manuscript received 23 May 1997; revised 2 June 1999; accepted 27 Aug. 1999.

Recommended for acceptance by S.R. Faulk.

For information on obtaining reprints of this article, please send e-mail to: tse@computer.org, and reference IEEECS Log Number 105117.

Authorized licensed use limited to: TU Wien Bibliothek. Downloaded on August 18, 2026 at 12:15:18 UTC from IEEE Xplore. Restrictions apply.

role of traceability [5], [6]. The source aspect remains important: Only trace objects available in *persistent* sources actually constitute long-term traceability, so traceability methods and tools have to make traces persistent reliably and at acceptable cost.

Studies at the management level usually focus on the **STAKEHOLDER** aspect (i.e., agents involved in the management of traceability). A good example is the work of Yu and Mylopoulos [7] who use dependencies between stakeholders as a starting point for driving and documenting requirements processes. The roles of stakeholders with respect to objects are emphasized in Gotel and Finkelstein's work on contribution structures [8]. Their empirical study shows the practical value of using a rich system of stakeholder roles in documenting the creation of requirements [9]; our longitudinal case study [10] shows that also the different usage roles are important in designing and implementing traceability systems. Summarizing, the stakeholder aspect becomes more important as the duration and complexity of projects grow.

Despite the importance of considering physical trace management and stakeholder roles, current practice shows that the efficiency and effectiveness of traceability support is largely determined by the system of **OBJECT** types and traceability link types offered. This is not only important for fine-grained change management, but also has repercussions for the two other aspects, i.e., for source management and stakeholder dependencies [11]. In this paper, we therefore focus on the analysis of object types and trace link types used in current practice and organize our reference models around them. However, after presenting the models, we shall discuss their embedding with the stakeholder and source management aspects, including the implications for tool design drawn from our studies.

1.2 Research Overview

Our own efforts to develop a reference model of traceability were driven by experiences with the deployment of REMAP, a model for design rationale and dependency management we developed in the late 1980s [12], [13]. The development of this model was guided by experiments with experienced software engineers and it has in fact served as a starting point for several commercial tool developments, including the Knowledge Based Software Assistant (KBSA) environment by Andersen Consulting. Despite this apparent success, some users of REMAP pointed out that it was too complex for some usage situations even though each of its features was needed *sometimes*. The informal feedback gained in these discussions was that a *broad variety of traceability strategies* is practiced in industry and the existing models are too simple and/or too rigid to deal with this variety.

In 1992, we therefore embarked on a two-pronged strategy. In the European NATURE project, we began to develop model-driven approaches to process guidance and traceability that would provide the technical means for dealing with a wide spectrum of traceability techniques. As a starting point for this activity, we developed a basic traceability framework that has become known as "the three dimensions of requirements engineering" [14], [15]. In [11], this framework was elaborated by a set of 18 types of

traceability links found in the literature and by the RSM model which synthesizes the objects to be traced from 21 international software engineering standards. In [16], [17], the resulting process-integrated tool environments are discussed.

In parallel, we began a four-year series of empirical studies with American software development organizations with the goal of

- capturing the current and desired practice in requirements traceability,
- developing and validating reference models which could guide an improved practice, and
- assessing the impact of good traceability on the software development and maintenance process.

This paper presents the reference models, together with their grounding in the empirical studies, and relates the findings to requirements for future traceability mechanisms. Companion papers discuss the organizational factors that guide the selection user organizations make among such models [18] and the organizational implementation and impact of traceability, based on a two-year case study in one specific organization [10].

In Section 2, we describe the design of the empirical studies, starting from a brief survey of traceability uses reported in the literature. Section 3 presents the first group of results, a simple meta model according to which traceability schemes can be organized. The studies show that there is a clear distinction between low-end and high-end users of traceability representing very simple and very comprehensive traceability practices (explained in detail in Section 2.5.2). Therefore, Sections 4 and 5 describe low-end and high-end reference models separately and indicate ways to migrate from one to the other. Section 6 contains a fairly detailed example on use of high-end models for supporting a large system-engineering project.

Sections 7 and 8 discuss the implications of our results. In Section 7, we define four basic categories of traceability link types and show how they map to the concrete link types in the reference models; by discussing available support for each link type, we relate our results to existing research. In Section 8, general conclusions concerning the functionality of traceability tools are drawn; the application of these conclusions is demonstrated by another case study, the development of the Design Rationale tool within the KBSA ADM system of Andersen Consulting. Section 9 summarizes our results.

2 EMPIRICAL STUDIES

In this section, we describe the empirical studies used for defining, validating, and applying the reference models.

2.1 Background: Definitions and Uses of Traceability

Requirements traceability has been identified in the literature as a quality factor—a characteristic a system should possess and include as a nonfunctional requirement [19]. Many standards governing the development of systems for the U.S. Government (e.g., MIL-STD-2167-A

and MIL-STD-498 which replaces it [20]) require the development of requirements traceability documents.

Edwards and Howell [21] define traceability as a technique used to “provide a relationship between the requirements, the design, and the final implementation of the system.” Palmer [22] states that “traceability gives essential assistance in understanding the relationships that exist within and across software requirements, design, and implementation.” These relationships allow designers to show that the design meets the requirements and help early recognition of those requirements not satisfied by the design. Palmer states that traceability helps ascertain “how and why system development products satisfy stakeholder requirements.” Hamilton and Beeby [23] view traceability as the ability to “discover the history of every feature of a system” so that the impacts of changes in requirements can be identified. Greenspan and McGowan [24] state that the ability to allow changes to any artifacts—requirements, specification, implementation—to be traced throughout the system is an important property of any systems description technique. In short, requirements traceability is a characteristic of a system in which the requirements are clearly linked to their sources and to the artifacts created during the system development life cycle based on these requirements.

Many different stakeholders—project sponsors, project managers, analysts, designers, maintainers, and end users—are involved in the system development life cycle. The traceability needs of these stakeholders differ due to differences in their goals and priorities and many problems of traceability stem from these differences in interest [25].

Stehle [26] identifies some criteria from the perspective of *managing* a system development effort. Traceability helps demonstrate that each requirement has been satisfied. Further, it helps ensure that each component of the system satisfies a requirement to ward off “gold-plating” [27], i.e., the addition of expensive and unnecessary features to a system.

In *requirements engineering*, a major challenge is the linking of rationales and sources to the requirements. Traceability facilitates communication between those involved in the project to alleviate these problems. Requirements management can be facilitated by capturing information necessary to understand requirements evolution and verification [28].

During *design*, traceability allows designers and maintainers to keep track of what happens when a change request is implemented before a system is redesigned [21]. Traceability is helpful if it can link designs to justifications, important decisions and assumptions behind them, and the contexts in which design solutions are arrived at [10].

Systems evolution requires a better understanding of the requirements which can only be achieved by tracing back to their sources [29]. Traceability provides the ability to cross-reference items in the requirements specifications with items in the design specifications [30]. Therefore, with complete traceability, more accurate costs and schedules of changes can be determined, rather than depending on the

engineer or programmer to know all the areas effected by these changes.

Last, but not least, *test procedures*, if traceable to requirements or designs, can be modified when errors are discovered [31].

2.2 Motivation and Overall Study Design

As a consequence of these different uses and perspectives on traceability, there are wide variations in the format and content of traceability information across different system development efforts. Even though some detailed proposals for traceability frameworks have been made [8], [11], the current literature and the standards do not provide detailed guidelines on what types of information must be captured and used in what contexts. Though a variety of tools used in the industry provide mechanisms to represent various types of linkages between system components, the interpretation of the meanings of such linkages is left to the user. Thus, there is a clear need for the development of reference models and guidelines for their use in software development activities. A comprehensive scheme based on such models can help ensure that traceability is maintained through all phases of the systems development process, from the requirements as stated or contracted by the customer, through analysis, design, implementation, testing, and operationalization of the product. This step, from the generic link types to their usage in typical development tasks as encoded in a reference model, requires empirical analysis.

As discussed earlier, providing traceability of requirements to their sources and the outputs of the system development process can be along several dimensions. Different stakeholders contribute to the capture and use of traceability information, often with different perspectives. For example, the types of links of interest to a user may be different from those of interest to a system designer. A user may be interested in the answer to the following question: What are the system components that are affected by a requirement? A systems designer may, in addition, be interested in the answer to the following: Why and how are the components affected by a requirement?

Further, the semantics of a given linkage as viewed by different stakeholders may differ. For instance, consider the following relationship maintained in a traceability table: REQUIREMENT LINKED-TO DESIGN object. An end user may view the relationship as a means to identify a design component that is generated by a requirement. A designer may view the same linkage as providing information on the constraint (i.e., requirement) that restricts a design object. The semantics or the meaning attributed to a linkage is guided by the reasoning that the user will be performing with the linkage. While the end user may be interested in just the connectivity information, the designer may be interested in the repercussions of changes in the solutions with the relaxation of the constraint (or redefinition of a requirement).

We conducted several empirical studies that capture the information needs (with respect to traceability) of different stakeholders involved in the software development process. Data collection methods included evaluation of major traceability tools, structured interviews with practitioners,

TABLE 1
Summary of Data Collection and Analysis

Stage		Data Sources	Outputs
Pilot Study		Six focus groups Seven verbal protocols Six structured interviews	Initial version of meta-model Detailed design for the empirical study
Analysis of current tools		Data on seven commercial traceability tools and three in-house traceability tools -including review of literature, vendor training, interviews with at least two major users of each tool	Capabilities and short-comings of current tools Understanding of various approaches to traceability support
Main Study	Phase I	Five focus groups of traceability practitioners One group of experts	Traceability meta-model Initial coding scheme for development of reference models
	Phase II	23 focus groups of traceability practitioners Evaluation of traceability documents	Low-end model of traceability High-end models of traceability
	Phase III	Two groups of experts drawn from different industries	Comments and informal "validation" of models
Model Use		Use of models in CASE tools Design rationale models in KBSA ADM Requirements management models in Tracenet High-end models in SLATE System Engineering Organizations	Case studies on the use of models Industrial uptake of results

focus groups involving the various stakeholders, and an in-depth case study of traceability practice in one organization. Due to the elaborate preparation required to assemble experienced practitioners in their organizational settings for these empirical studies, data collection spanned a period of over three years. Subsequently, the models developed in this research have been used and incorporated in a number of prototypical and commercial tools.

Table 1 summarizes the various steps in our data collection and analysis steps and identifies the major outputs of these phases. Each of these steps will be described in the following subsections.

2.3 Pilot Study

The pilot study determined the appropriateness of the various data collection approaches and guided the development of relevant instruments, such as a questionnaire for structured interviews and focus groups. It involved focus groups for idea generation, protocol analysis to study problem-solving behavior, and structured interviews to evaluate the initial results.

A *focus group* is a semistructured exchange among a small group of people conducted with a clear agenda. This technique is the most commonly used qualitative data collection methodology in market research [32].

Protocol analysis is commonly used in building expert systems. The thought process of subjects is captured through think-aloud verbal protocols. The protocols are recorded as subjects think through a problem and verbalize their thoughts. The recorded protocols are analyzed to capture knowledge elements and operators that characterize the problem-solving behavior. In our context, the

knowledge elements provide insights into the relationships or linkages and the operators provide insights into the reasoning performed with the relationships.

Structured interviews were designed to provide further insights into the nature of the traceability links, as well as potential areas for providing support by explicit representation of these relationships. Further, various types of reasoning performed by the stakeholders were identified such that mechanisms can be developed to aid the tasks of design and maintenance based on the nature of the relationships and the perceived areas of potential benefits.

The pilot study involved 58 master's students in information technology at a graduate university. Many subjects had extensive experience in domains such as ship building and aviation maintenance, where traceability is widely practiced. The participants had also completed a comprehensive project involving the analysis and design of a segment of a large-scale, real life system.

Six focus groups, consisting of eight to 10 subjects each, were conducted. The discussion was moderated to discuss definitions, need for traceability from different stakeholder perspectives, the issues involved in creating a comprehensive traceability practice. Seven subjects participated in the protocol analysis phase of the project. Think-aloud protocols of subjects involved in defining traceability links (among requirements, design elements, and implementation) in projects they had participated in were collected. Six subjects participated in structured interviews where the results developed from the above data collection sessions were reviewed and elaborated.

Besides the development of initial versions of our meta-model, the primary outcome of the pilot study was the

development of a detailed design for the empirical study. Focus groups and structured interviews were identified as the primary methods for data collection during the subsequent phases. The importance of incorporating both contributors and users of traceability information in the study was also highlighted.

2.4 Analysis of Current Tools

As a first step toward developing a clear understanding of the current practice of traceability, a detailed study of the capabilities of major tools that support traceability was undertaken. All leading CASE tools on the market were included in the study. These include SLATE, DOORS, Marconi RTM, Teamwork/RQT, RDD-100, RTS, and RTrace.¹ Three CASE tools developed by system engineering organizations for in-house use were also studied. The study included a detailed review of vendor-provided literature, participation in vendor-provided training (ranging from one to five days) on the tools, and interviews with at least two major users of each of the tools. It provided clear understanding of the capabilities of current tools, the areas of traceability supported by each tool and shortcomings of the different approaches to traceability supported by them, which was very valuable in the design of focus groups and interviews. The details of the tool analysis are not reported here, as they were subsumed by a later survey of traceability tools conducted by the International Council on Systems Engineering (see www.inco-se.org/workgrps/tools/tooltax.html) and extended in [16].

2.5 Main Study

The main part of the study comprised 30 focus group discussions in 26 organizations. They were conducted in a wide variety of industries, including defense, government, aerospace, hardware development, pharmaceuticals, utility, system integration, electronics, and telecommunications. Table 2 gives an overview of the organizations involved. Whenever feasible, participants representing different user and consumer perspectives were included. On average the focus groups consisted of five participants. As in the pilot study, focus groups were followed up with selected structured interviews.

The participants had experience in several key areas of systems development including project management, software engineering training, requirements management, software testing, system integration, configuration management, procurement, development support, software quality assurance, systems analysis, maintenance, software implementation. The participants had an average of 15.5 years experience in systems development.

2.5.1 Phase I

The focus groups were conducted in two phases. In Phase I, building on results of the pilot study, data from five focus groups involving 30 participants was used in developing a

traceability meta-model. The meta-model defines the language in which traceability reference models are described. It provides the primitives to represent different traceability linkages among the various objects produced during systems development. It further represents the agents involved in the systems development process, as well as the sources in which traceability information is captured. The meta-model is discussed in Section 3. The meta-model was checked for plausibility and terminology by a group of experts drawn from different industries and stakeholder groups.

An important outcome of this study is also the identification of various types of traceability information and the development of an initial scheme for encoding reference models from data collected in subsequent phases.

2.5.2 Phase II

The second phase involved the development of reference models of traceability practice. Data from 23 focus groups was used in the development and classification of traceability links representing current practice and ideal practice.

Issues addressed in the focus groups. Each focus group started with clarifications on the definitions and terms used. This included the identification of the various stakeholders involved in traceability, the system lifecycle phases covered in the project being discussed, the system components to which traceability is intended, and the various views on traceability.

The focus group participants were then asked to discuss their current traceability practice as well as desired traceability practice. To achieve comprehensive understanding of the current and desired practices, participants were asked to elaborate their ideas from two different perspectives:

- *User perspective.* What types of traceability information are needed and are useful at different phases of the system development process? How is this information captured and by whom and at what phase of the system development life cycle? What are the costs associated with the capture of this information?
- *Provider perspective.* What types of traceability information does each stakeholder capture? In what phase of the lifecycle and how is it used and by who and when? What are the benefits associated with the capture of this information?

Wherever available, the participants were asked to answer these questions with specific reference to the reports and documents produced and/or used in the organization.

The next part of each focus group was devoted to a discussion of the different types of traceability links used in that organization:

1. How are the various traceability relationships identified in the traceability documents?
2. How are the traceability relationships created/defined and prioritized? Who are the stakeholders involved in managing these links?

1. System Level Automation Tool for Engineers (SLATE) is a Computer-Aided System Design Groupware and is a trademark of TD Technologies, Inc. DOORS is a trademark of Quality Systems and Software Limited. RTM is a trademark of Integrated Chipware/Marconi Systems Technology. Teamwork/RQT is a trademark of Cadre Technologies, Inc. RDD-100 is a trademark of Ascent Logic Corporation. RTS is a trademark of Electronic Warfare Associates, Inc. RTrace is a trademark of Transfer Logic Corporation.

TABLE 2
Organizations Involved in Focus Groups and Interviews

No	Area of Business	Stakeholders involved in study	Project Size (cost, number of requirements)	Traceability focus
1	Computer Hardware, System Integration	system engineers project manager quality assurance, system analysts	over 1 million lines	Requirements management, Satisfying requirements, Contract compliance, Project monitoring and control, Rationale, Change control
2	US Government System Development Organization	project sponsors project manager lead system engineers	multi--billion dollar	Rationale, Project monitoring, Satisfying requirements, Resource management
3	US Government System Development Project Management Organization	project sponsor, contractor engineers, system analysts	several hundred million dollars	Standards compliance, project tracking and control, resource management, change control
4	US Government System Development Organization	systems analysts, project sponsors	1400 requirements	requirements evolution, standards compliance
5	US Government System Testing Organization	test engineers, manager, contractor engineers	1300 requirements	satisfaction of requirements, standards compliance, development of appropriate verification procedures
6	System Integration	system engineers, test engineers, maintenance engineers	over 100 million dollars	satisfaction of requirements, inter-component dependencies, development of appropriate verification procedures, rationale
7	Pharmaceuticals	system engineers, maintenance engineers	over 1000 requirements	satisfaction of requirements, rationale, quality assurance, requirements allocation
8	Utility	project manager, system engineers, maintenance engineers	approx. 15 million dollars	satisfying requirements, change control, project tracking and monitoring, compliance with regulations
9	Telecommunications	project manager system engineers, test engineer	6000 requirements	satisfying requirements, quality assurance, project tracking and monitoring
10	Aerospace	project lead engineer, system engineer	several hundred million dollars	requirements allocation, requirements management (evolution), requirements rationale
11	Aerospace	system engineers, maintenance engineer	over 5000 requirements	requirements allocation, design rationale, component dependencies, resource monitoring

- How are the traceability links maintained, reviewed, updated, or changed, by whom, and during which activities?
- What are the mechanisms for defining and maintaining the semantics of these links?
- What are the uses of each type of traceability link identified?
- What are the issues that facilitate or impede the implementation of traceability?

TABLE 2 (continued)
Organizations Involved in Focus Groups and Interviews

No	Area of Business	Stakeholders involved in study	Project Size (cost, number of requirements)	Traceability focus
12	Electronics	system engineers, project manager, maintenance engineers	multi-million dollars	quality assurance, satisfying requirements, standards compliance, design rationale
13	Defense Contractor, System Integrator	system engineers	over 25 million dollars	requirements allocation, satisfaction of requirements, requirement management (evolution)
14	Electronics	system engineers	600 requirements	satisfaction of requirements
15	Contractor for Financial Services	system engineers	500 requirements	satisfaction of requirements
16	Govt. contractor	system engineers	900 requirements	satisfaction of requirements, contract compliance
17	Govt. project Management organization	project managers, systems analysts	over 100 million dollars	project tracking and control, contracts compliance, standards compliance, resource management
18	Govt. project sponsor	project sponsors project manager system analysts	over 150 million dollars	development of requirements compliance with standards, mission/organizational needs, project costing/estimating
19	Automobile	system engineers, maintenance engineers	multi-million dollars	requirements allocation, satisfaction of requirements, resource monitoring, rationale
20	Electronics	test engineers, project manager, systems analyst	1300 requirements	satisfaction of requirements, development of appropriate verification procedure
21	Govt. contractor	system engineers	400 requirements	development of traceability documents required by sponsor (requirements-test matrix)
22	Contract Software Developer	system engineers	1700 requirements	development of allocation and test matrices
23	Government Contracting Organization	contract specialists, project manager	over 10,000 requirements	contract compliance, standards compliance, project monitoring and control
24	Contract software Development and Maintenance	Maintenance engineers	800 requirements	requirements satisfaction, test plan development
25	Aerospace	Configuration management personnel, quality assurance, system engineers	8000 requirements	requirements evolution, system /component evolution, requirements satisfaction
26	Telecommunications	system engineers, systems analysts	6000 requirements	development of requirements, requirements rationale, project costing/estimating

TABLE 3
Characterization of Low-End and High-End Usage of Requirements Traceability

Characteristic	Low-end traceability user	High-end traceability user
Number of organizations in the study	Nine	Seventeen
Number of participants	Fifty-four	Eighty-four
Typical complexity of system	about 1000 requirements	about 10000 requirements
Traceability experience level	zero to two years	five to ten years
User definition of traceability	documents transformation of requirements to design	increases the probability of producing a system that meets all customer requirements and will be easy to maintain
Main application of traceability	requirements decomposition requirements allocation compliance verification change control	full coverage of life cycle, including user and customer; captures discussion issues, decisions and rationale; capturing traces across product and process dimensions

Each focus group session lasted between 1.5–2.5 hours. All sessions were audio or video-taped for further analysis. When this was prohibited by security or confidentiality concerns, a scribe documented the discussions.

During the focus groups, a deeper understanding of the different types of traceability links discussed in the literature was developed. The usage of these link types in practice was gradually encoded in reference models for the major uses of traceability, as discussed in Section 5.

High-end and low-end users. It quickly became apparent that the participants in our study could be categorized into two distinct groups with respect to their traceability practice. We refer to them as low-end and high-end traceability users.

The characteristics of both groups are summarized in Table 3. The representation of the two groups in our empirical study is also discussed in this table. Briefly, low-end users with just a few years of traceability experience see it simply as a mandate from the project sponsors or for compliance with standards. In contrast, experienced high-end users are experiencing traceability as a major opportunity for customer satisfaction and knowledge creation throughout the systems lifecycle. The perceived importance of traceability grows with system complexity. As the following sections will show, the reference models used for low-end and high-end traceability users differ substantially. However, it should be noted that even an organization with predominantly low-end practice may have “mature” traceability practice in some areas and vice-versa.

Development of reference models. The analysis of the data followed standard procedures used in qualitative research that employs focus groups and interviews. The data was analyzed using a form of content analysis [33] where the data is categorized into concepts. The primary concepts in our study are the various types of traceability links used in current and ideal practice. The results presented in this paper are based on a “qualitative or ethnographic summary” [34] and rely on direct quotations from the data. As suggested by Morgan [34], the analysis

was done in two phases. During Phase I, a detailed examination of the data from five focus groups provided the initial categories and concepts along which the data was coded. This scheme was developed iteratively so that it was sufficient to explain the data. The scheme from this study was used in the analysis of data from Phase II to identify and refine components of traceability models. Triangulation [33] across multiple data sources (multiple participants that represent different stakeholder perspectives from each organization) and across data collection methods (focus groups, interviews, and review of traceability documents) helped confirm the findings. Each concept represented in the reference models presented in Section 5 was included only if it was observed in practice (as reported as current practice in focus group discussions and/or structured interviews and/or represented in traceability documents) and identified by others as a part of desired practice. Each traceability information in the high-end models is either in use or considered desired practice by at least six organizations in our study.

We have developed two sets of models: The low-end models describe the current practice in low-end organizations. The reference models for high-end users in different systems development activities represent the best practices in the industry.

2.5.3 Phase III

The reference models developed from Phase II were presented to two groups of experts for comments and feedback. These two groups were composed of experienced traceability users drawn from different industry groups. The two groups provided informal “validation” of the models presented here and further provided suggestions on the implementation of the models in system engineering environments.

2.6 Model Use in Tools

Based on the suggestions received from the experts and our analysis of the current traceability tools, we identified characteristics of system engineering environments for the

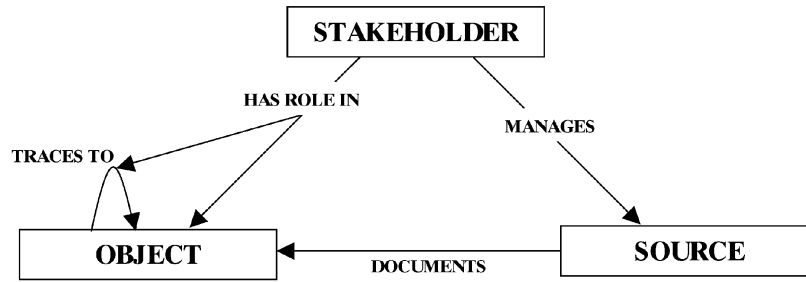


Fig. 1. Traceability meta model.

implementation of the traceability reference models (cf. Sections 7 and 8). To develop and adapt reference models, the software information system requires support for specialization and instantiation, applied both to nodes and links.

In order to ensure formal consistency of the proposed models and to be able to demonstrate them to users and tool vendors, the meta model and the reference models were first encoded in ConceptBase [35], a knowledge-based meta database management system based on the Telos language [36]. We shall use ConceptBase screenshots to show the reference models in Sections 4 and 5.

For initial validation, a ConceptBase prototype of an early submodel was included in the concept demonstrator of the Knowledge-Based Software Assistant (KBSA) project of the U.S. Air Force. Demonstrations of these prototypes, together with their empirical grounding, led to the adoption of our reference models in several commercial traceability tools:

- The traceability scheme of SLATE, a leading system engineering tool, is architected around earlier versions of the models presented here.
- The design rationale models presented here are the primary components of the Knowledge-Based Software Assistant tools Concept Demo and ADM created by Andersen Consulting on behalf of the U.S. Air Force.
- The models have been incorporated in the Tracenet tool that supports the capture of traceability across multiple environments.
- The models have been tailored and used in a large commercial and a government system engineering organization [10]. The models have also been tailored to support traceability to formal specifications in a model management system [37].

Though a detailed discussion of the experiences from these implementations is beyond the scope of this paper, they seem to provide strong evidence of usability and scalability of the models in real-life system engineering. We present two case studies in Sections 6 and 8.

3 A SIMPLE META MODEL OF REQUIREMENTS TRACEABILITY

In the next three sections, we present the reference models resulting from our studies. We assume that our traceability reference models will be implemented in some trace

repository (manual or computerized). It is widely accepted that such a repository will comprise at least three layers (for details, see e.g., [38], [39], [40], [11]):

- the meta model defining the language in which traceability models can be defined;
- a set of reference traceability models which can be customized within the scope defined by the meta model; and
- a (possibly distributed) database of actual traces, recorded under the chosen models.

We adopt the convention that we denote node meta-classes (e.g., **STAKEHOLDER**) by small bold caps and their instances (e.g., CUSTOMER) by nonbold small caps. Similarly, link meta-classes (e.g., *TRACES-TO*) are denoted by bold italics, specific link types by standard italics (e.g., *REFINES*).

The practitioners and focus groups in Phase I of the main study confirmed that the most essential aspects of traceability can be captured in the very simple meta model, shown in Fig. 1, which thus provides the basic language primitives for categorizing and describing traceability models in more detail.

Each entity and link in the meta model can be specialized and instantiated to create organization or project specific traceability models. The meta model can be used to represent the following dimensions of traceability information (cf. Table 4):

1. **What** information is represented—including salient attributes or characteristics of the information?

In the model, **OBJECTS** represent the inputs and outputs of the system development process. Examples of various types of **OBJECTS** include REQUIREMENTS, ASSUMPTIONS, DESIGNS, SYSTEM COMPONENTS, DECISIONS, RATIONALE, ALTERNATIVES, CRITICAL SUCCESS FACTORS, etc. These represent the major conceptual elements among which traceability is maintained during the various life cycle stages. **OBJECTS** are created by various organizational tasks. Examples of tasks include systems analysis and design activities. This information can be represented as an attribute of **OBJECTS**.

Traceability across various **OBJECTS** is represented by the *TRACES-TO* links. For example, a *DEPENDS-ON* link between two objects (a REQUIREMENT and an ASSUMPTION) can be represented as a specialization of this *TRACES-TO* link.

TABLE 4
Traceability Dimensions—An Example

DIMENSION	EXAMPLE
What?	Rationale for Design Decisions
Who?	Systems Designer
Where?	In the design documentation library
How?	Using Tool X; Represented as the "Rationale - justifies - Design Decision traceability link"
Why?	To facilitate understanding and communication with other designers, maintainer; to avoid rework
When?	At the finalization of the design

2. **Who** are the **STAKEHOLDERS** that play different roles in the creation, maintenance and use of various **OBJECTS** and traceability links across them?

In the model, **STAKEHOLDERS** represent the agents involved in the system development and maintenance life cycle activities. Examples of **STAKEHOLDERS** include the project managers, systems analysts, designer etc. These **STAKEHOLDERS** act in different **ROLES** or capacities in the establishment and use of the various conceptual **OBJECTS** and traceability links.

3. **Where** it is represented—in terms of sources that “document” traceability information?

All **OBJECTS** are documented by **SOURCES**, which may be physical media, such as documents or intangible things, such as references to people or undocumented policies and procedures. Examples of **SOURCES** include REQUIREMENT SPECIFICATION DOCUMENTS, MEETING MINUTES, DESIGN DOCUMENTS, MEMORANDA, TELEPHONE CALLS as well as references to various **STAKEHOLDERS** using their phone numbers, e-mail address etc., **STAKEHOLDERS** manage the **SOURCES**; i.e., they create, maintain, and use them.

4. **How** this information is represented—both by formal and informal means and how it relates to other components of traceability?

The sources, as mentioned above, can be physical or intangible. Further, they can be represented at different levels of formality. Some sources such as requirements specifications may be text documents, whereas others design documents may be represented in multiple formats such as graphics and text.

5. **Why** a certain conceptual **OBJECT** was created, modified, or evolved?

The rationale behind the creation, modification and evolution of various conceptual **OBJECTS** can be represented as a specialization of the meta-class **OBJECT**. Then, it can be linked to the conceptual object (using a specialization of the traces-to link). More complex models of rationale, which include issues, alternatives and arguments supporting and opposing them can also be represented as specialization of the **OBJECT-TRACES-TO-OBJECT** relationship in our model.

6. **When** this information was captured, modified, and evolved?

Relevant temporal information about any of the entities or links in our model can be represented as their attributes. For example, the frequency or the time/duration at which a requirement or design was created, reviewed, modified, or justified by a specific rationale can be represented with this scheme.

The three nodes of the meta model correspond roughly to the three dimensions of requirements engineering proposed by Pohl [15] in that they cover the aspects of understanding (objects), agreement (stakeholders), and physical representation (sources). However, note that we are not discussing the requirements process per se, but the creation and usage of traces.

4 LOW-END USE OF TRACEABILITY

A model of the typical low-end user’s traceability efforts is provided in Fig. 2. The names of objects and links presented here are our interpretations (validated by the expert groups) of what was observed in practice. Many organizations simply use traceability tables to link various components of information without explicit identification of the semantics of such relationships. Low-end users created traceability links to model requirement dependencies, allocation of requirements to system components, compliance verification, and change control. For the following, recall that we denote traceability linkages by italic small-caps (*LINKAGES*), while components that they link are indicated with upper-case letters (**OBJECTS**). For every link in the model an inverse may be defined.

Typical low end users viewed requirements traceability as providing a link from initial REQUIREMENTS to the actual system COMPONENTS that satisfy those requirements. However, first, the higher level requirements must be decomposed to a more refined level. During this recursive process, lower level REQUIREMENTS are *DERIVED* from higher level system REQUIREMENTS. (“Often derived requirements are the problem; you do not know why and how they got there, want to make sure that every low level requirement is tied to the bigger level specification.”)² Typically, this

2. This is a direct quote from a subject participating in a focus group or interview. Henceforth, all quotes from a subject will be italicized and included within quotation marks, but no specific reference will be made.

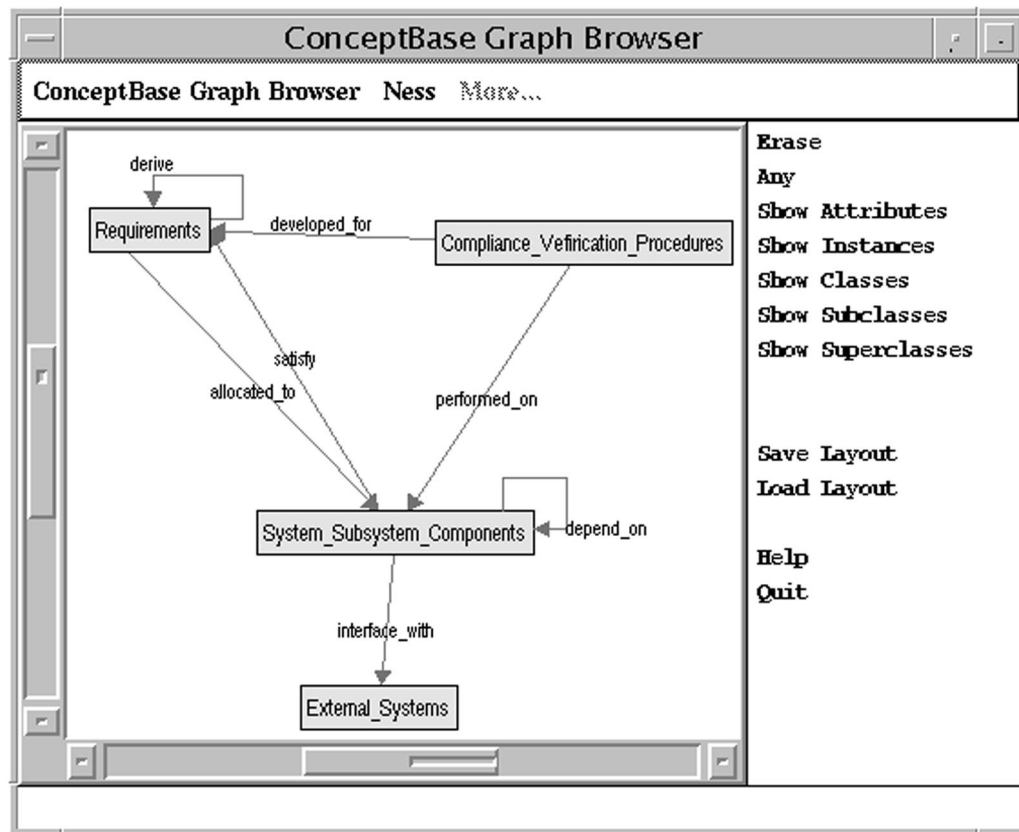


Fig. 2. Low-end traceability model.

information is captured in a relational database, and used in the form of a traceability matrix.

Original and derived REQUIREMENTS are *ALLOCATED* to system COMPONENTS. ("We want to track which component will take care of what requirement by creating an allocation table.") An allocation table is the common mechanism used to maintain this information. This simple two way mapping between requirements and system components is usually used also to ensure that there are COMPONENTS in the system that *SATISFY* all REQUIREMENTS. ("When you want to know which part of the system satisfies what requirement, you can go to the allocation table to identify it. Whether it does indeed or not is another matter—for testing to worry about.")

By capturing which components satisfy various requirements and which requirements are mapped to different components, the designer is able to verify that all requirements are addressed by the system.

In the Compliance Verification phase of systems development, low-end users use the requirements database, which contains the most current version of the system's validated requirements, to develop the system COMPLIANCE VERIFICATION PROCEDURES (CVPs) such as tests or simulations. CVPs developed for each REQUIREMENT are usually maintained in a REQUIREMENTS-and-TESTS traceability matrix. ("A matrix showing the tests to requirements is very important to make sure all requirements are adequately taken care of.")

If a change should occur in the requirements, then the traceability links could identify the CVPs that must be modified or redeveloped. CVPs are *PERFORMED* on the

system components verifying that the component satisfies the requirements. Results of the tests are used to verify that the system works and that it meets all of the requirements. ("Tests performed indicate whether we indeed achieved success in implementing goals and satisfied the requirements.") A system COMPONENT may *DEPEND* on others and may also *INTERFACE* with EXTERNAL SYSTEMS. This information is used in evaluating how a requirement is satisfied by a system component.

Low-end users lacked especially in the area of capturing rationale. In requirements management, for example, information concerning requirement issues, how they are resolved, and the rationale for the decisions is seldom captured. Likewise, similar information is absent in the design and implementation phases. ("Often we have no idea who made these decisions, and how they impact the rest of the effort. Simply trying to do these at the end of the project or after the fact does not work. Often the people who worked on it are gone without a trace, of what happened... not disciplined enough to document these... with all the demands on the team.")

5 HIGH-END USE OF TRACEABILITY

High-end users of traceability employ much richer traceability schemes than low-end users and also use traceability information in much richer ways. We have, therefore, segregated the high-end model into four parts for clarity: Requirements Management, Design Allocation, and Compliance Verification. In addition, Rationale Management now takes center stage.

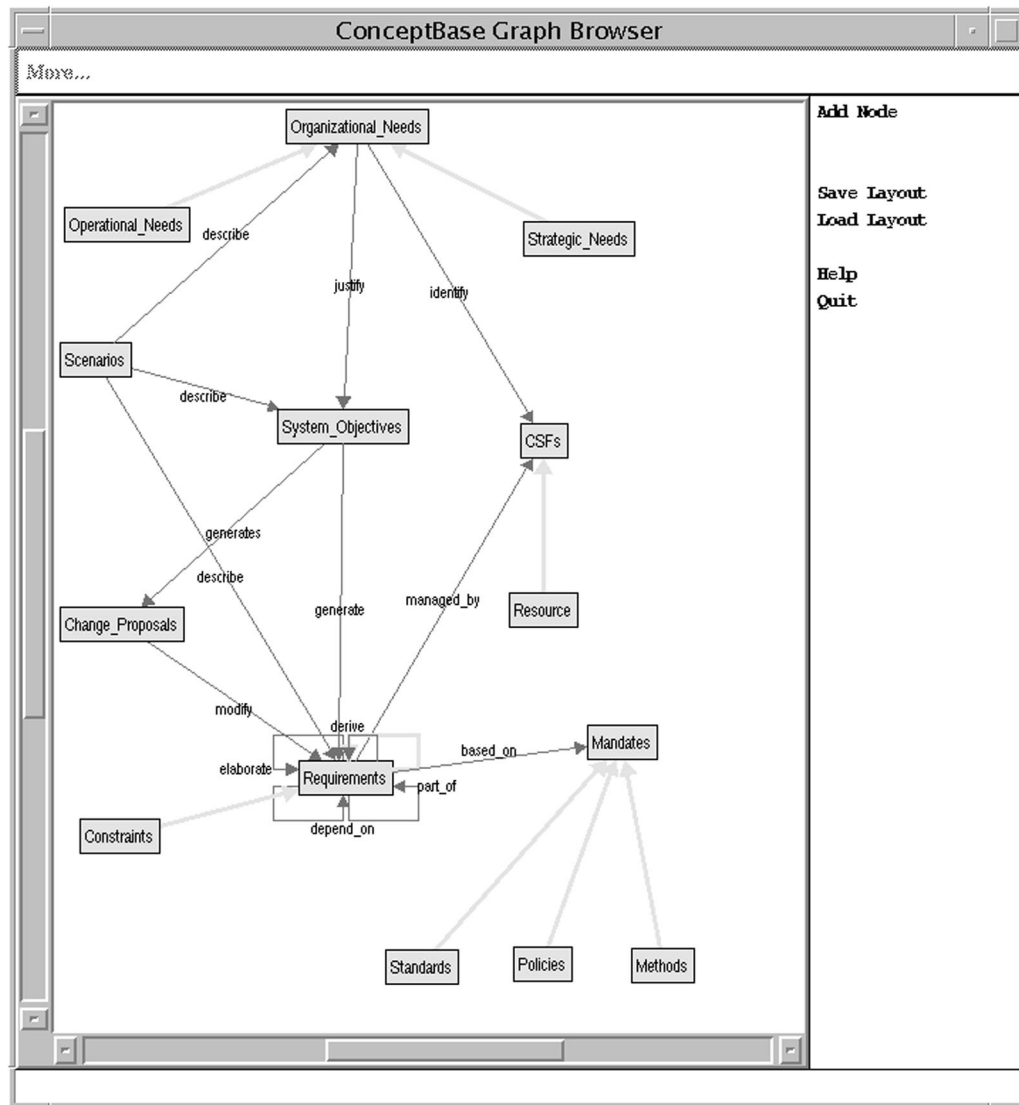


Fig. 3. Requirements management submodel.

5.1 Requirements Management Submodel

Traceability, when implemented correctly, would greatly benefit requirements management, facilitating requirements understanding, capture, tracking, and verification. The following is a discussion of the Requirements Management model presented in Fig. 3.

Systems are built to primarily satisfy ORGANIZATIONAL NEEDS. These needs may either be long term STRATEGIC NEEDS or more immediate OPERATIONAL NEEDS, as denoted by the IS-A hierarchy (shown as thick lines in Figs. 3, 4, 5, and 6). These two needs support each other and are often detailed in SCENARIOS DESCRIBING the intended use or purpose of the system. ("We build the system obviously to satisfy some long term or operational goals of our sponsors. In our environment, these are clearly stated in the mission needs statement—MNS—operational requirements document—ORD—two important documents to work within any effort. Obviously, these are at a very high level, laying out the goals.")

The objectives that the system is trying to meet, SYSTEM OBJECTIVES, must be JUSTIFIED by ORGANIZATIONAL-NEEDS. ("...the system draws its legitimacy from the MNS and

ORD.") Stakeholders such as customer, program manager, program sponsor, etc., specify the SYSTEM-OBJECTIVES. The REQUIREMENTS for the system are GENERATED from these SYSTEM OBJECTIVES. ("The high level goals are after all too vague to be called requirements—for contracting. Specific, detailed requirements have to be developed to proceed further.")

Due to the large number of requirements associated with complex systems, it is important to determine those factors that are critical to the success of the project and to manage requirements based on how they contribute to these factors. The stakeholders involved in the identification of the ORGANIZATIONAL NEEDS also IDENTIFY these CRITICAL SUCCESS FACTORS (CSF). ("Stakeholders have some issues or values like, no mistakes or system errors, performance is the most important thing, the power requirements are the most critical item in this area.") RESOURCES such as Cost, Time, Weight, Voltage, etc. are examples of CSFs. ("A lot of decisions have been made... because of cost and weight, mostly cost.") ("It will be a combination of technical and cost trades.") REQUIREMENTS for the system are MANAGED (budgeted, monitored, tracked, etc.) by these CSFs. For example, the mission

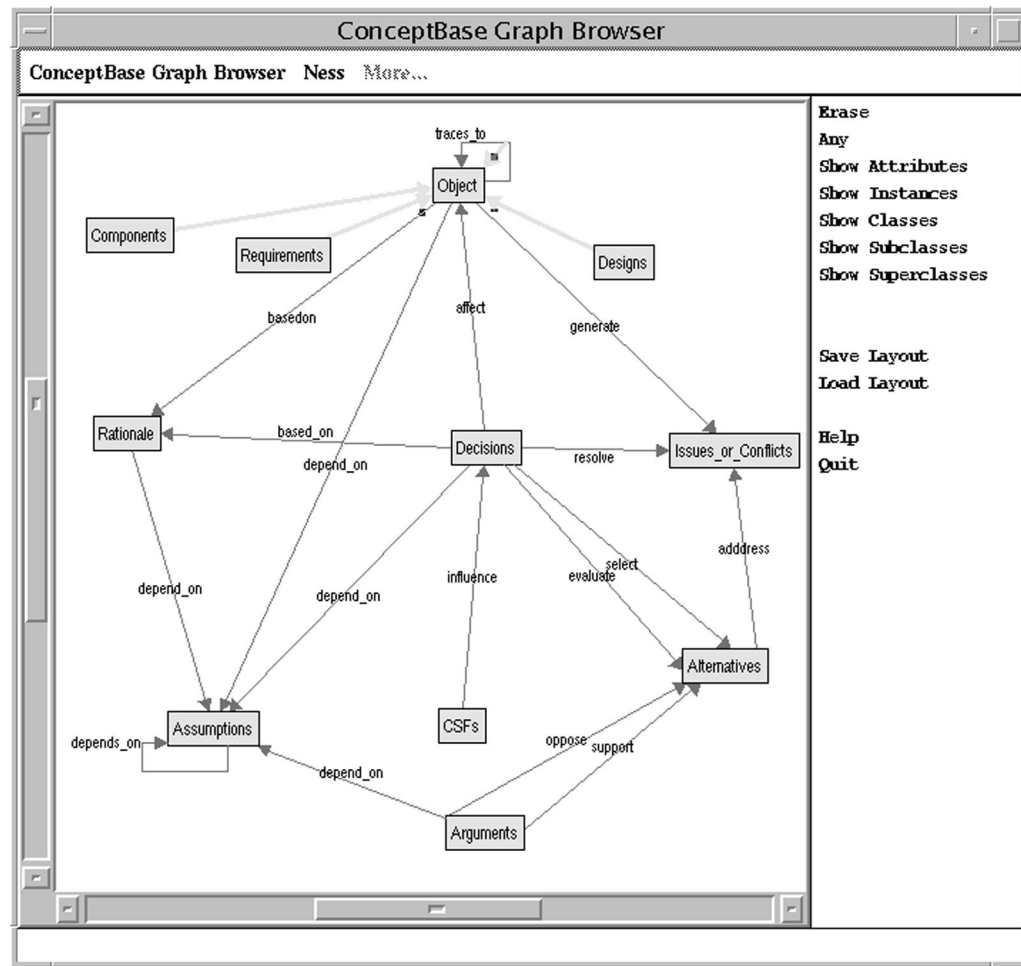


Fig. 4. Rationale submodel.

criticality (a CSF) of requirements could be used in classifying and monitoring requirements. Similarly, CSFs such as weight in an aerospace system or the total cost of development can be used in tracking and monitoring requirements. As part of the negotiation process among stakeholders, many trade-offs are made in deciding the scope and functionality of the system depending on their impact on CSFs. ("Typically we enter into a complex negotiation process of what is in scope, what is out of scope; what is affordable, what is not affordable. And I believe this is where traceability earns its keep.") Such decisions determine, among many things, whether requirements identified initially are feasible, cost effective, or desirable.

Not all requirements are equal in significance or criticality; requirements may be traced through the lifecycle at different levels of granularity or detail in order to optimize on resources spent. It may be unnecessary or even undesirable, considering the overhead involved in maintaining traceability, to maintain linkages between every requirement and every output created during the systems design process. The CSFs may be used in deciding when it is essential to trace requirements in detail and when fine-grained traceability is less important ("...just try to get as much traceability information for the critical requirements, and not as much for the requirements that may not be as important.")

REQUIREMENTS can be traced throughout the lifecycle to

provide stakeholders with a view to understand and evaluate whether the system supports these CSFs ("...the smaller subset is the driving, high risk set of requirements that merit more visibility in the tracking of the program... we have an attribute of a requirement that we call technical performance measure (TPM). Is it high risk, do we want to put a TPM against this requirement? The TPMs are all programmed, tracking, giving them higher visibility to see how we are doing.") Further, requirements may also be based on STANDARDS, POLICIES, and PROCEDURES. Maintaining the links between requirements and the sources and the requirements will help understand and correctly interpret them.

CONSTRAINTS may be treated as a type of REQUIREMENT (denoted by *IS-A* link). Several focus groups stated how constraints become hard requirements because they set limits within which the system is to be developed. ("Systems are designed and decisions made with constraints considered.")

5.1.1 Evolution

One of the biggest challenges in managing large, complex systems is due to the way requirements are constantly evolving and changing. Each focus group noted that, in order to accurately reflect this volatility, a requirements traceability scheme should help document and understand the evolution of requirements. Two major sources of

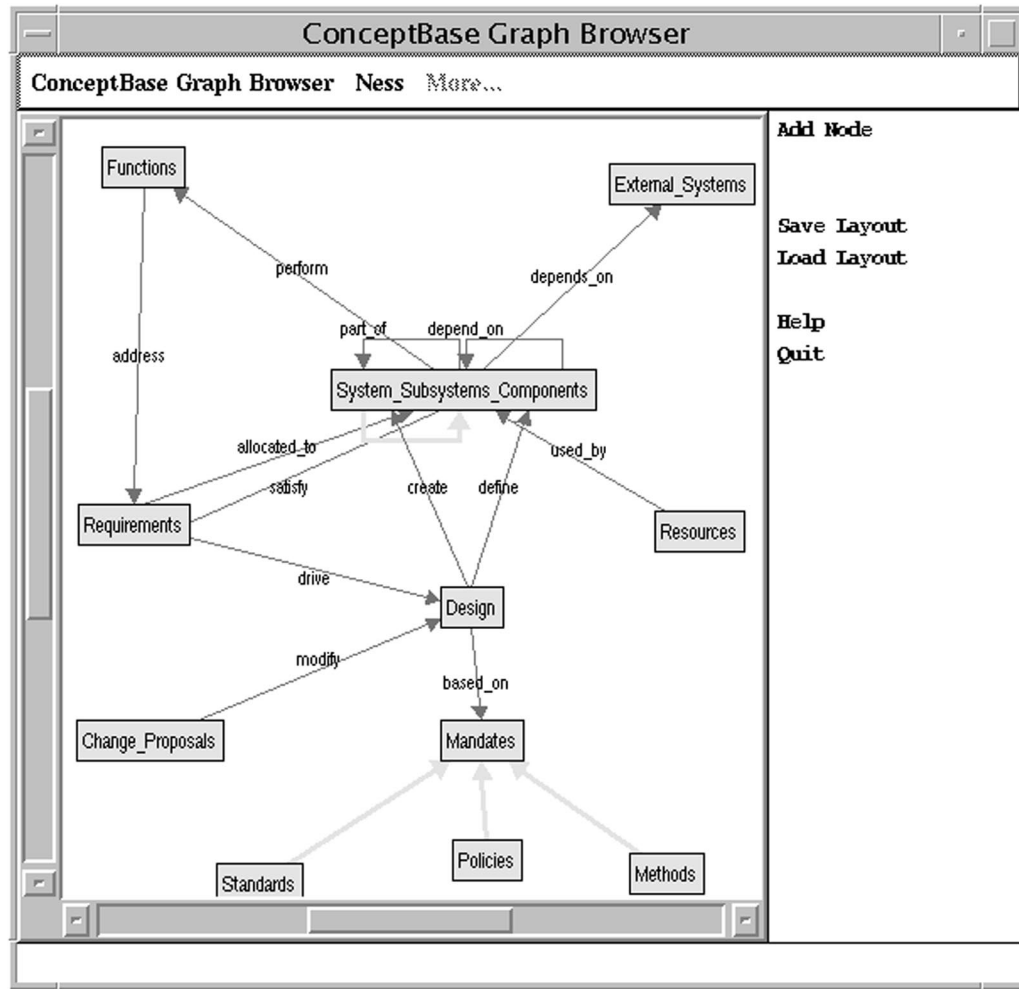


Fig. 5. Design allocation submodel.

changes are: 1) the need to fix a deficiency in the system, identified during, say operations or testing (discussed in detail in Section 6.4) and 2) changes in ORGANIZATIONAL NEEDS. Various stakeholders modify SYSTEM OBJECTIVES based on changed ORGANIZATIONAL NEEDS, leading to changes in REQUIREMENTS. In projects with very long life cycles, such as military applications, not only is the nature of systems requirements dynamic, but even the ORGANIZATIONAL NEEDS (mission and operational) and the SYSTEM OBJECTIVES keep evolving. ("Systems that were considered successful during the cold war era are no longer considered useful as strategic and mission needs are evolving rapidly.") It is not possible, however, to abandon all investments in such systems and develop new systems to meet current needs. The lack of comprehensive traceability is a primary reason for the inability to identify components that are linked to various objectives and modify them to meet the current requirements. ("We cannot clearly figure out which part of the system we can retain and which part is no longer relevant... we do not have the traceability details for these systems any more.")

Traceability links are needed among REQUIREMENTS that get specified at different levels of detail as they evolve through the various stages of the development lifecycle. The recursive links among REQUIREMENTS shown in our model are useful in providing a historical record of requirements

evolution and in mapping a REQUIREMENT back to its sources, thereby facilitating a complete understanding of where a requirement comes from.

5.1.2 Derived Requirements

Lower level REQUIREMENTS are DERIVED from higher level REQUIREMENTS, usually based on assumptions made by stakeholders. Derived requirements need to be tracked carefully as they are likely to be a major source of conflicts and issues, and are subject to changes as assumptions behind them are often unrecorded. Requirements that are derived "create a collection of requirements at a different level" of detail. Traceability helps clearly identify derived requirements that "partially satisfy the originating requirement." ("We find things in requirements documents which most people did not know where they came from. Then we will figure out that someone created these consolidating or clarifying other requirements. We now have a policy to identify—and before that—agree on these derived requirements. There is lot of room for problems when we go from customer given requirements to lower levels based on our interpretations. We try to document these also in as much detail or at least know who is responsible.") Further, this traceability information is useful in managing the "explosion" in the number of requirements ("...if you can start with 30 pages of requirements, top level, then you can go to your prime item

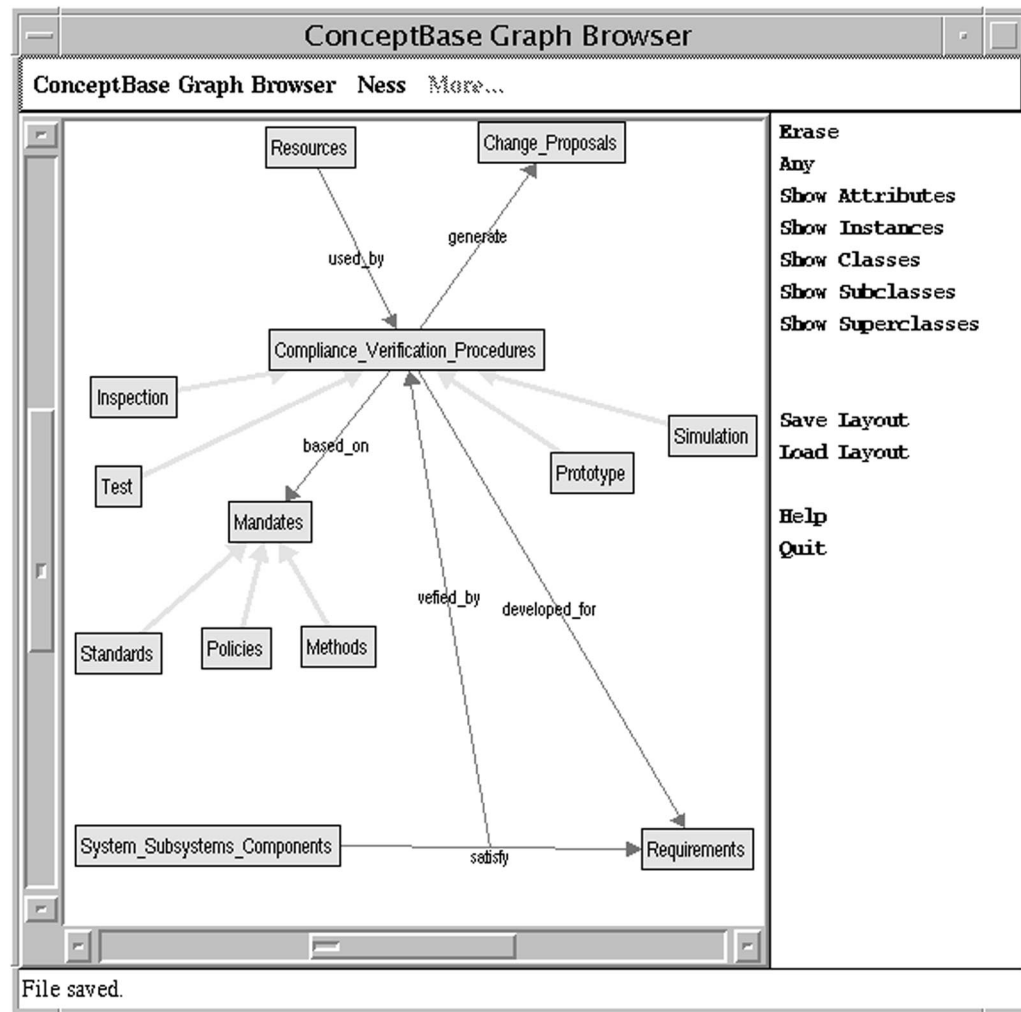


Fig. 6. Compliance verification submodel.

development specifications and you could consider those requirements derived from those 30 pages, but then you have those second and third generation... derived requirements between those... you have an explosion of derived requirements and relationships.")

Some REQUIREMENTS are ELABORATED by others, providing further explanation or clarification. This added detail assists in understanding the assumptions behind requirements as well as how they are interpreted. ("Sometimes you add requirements really because you find them unclear. They are refined, and become more specific in detail then in fact are more demanding than the original requirement might have been.")

REQUIREMENTS also DEPEND-ON others. For example, a requirement to use a specific software platform may depend on a requirement to use a specific hardware platform. Identifying these dependencies is important, especially when requirements change, so that the impact of the change on the entire system can be determined. ("Different people and teams work on different parts of requirements, some even from other parts of the organization or subcontractors. It is when it is difficult to trace whose part affects which other. After all they are part of the same system. We get into major problems down the line when we are unable to figure out these connections.")

Complex requirements are often broken down into their components, identifying simpler requirements that form a PART-OF them. These links help in understanding how various pieces of a requirement fit together and are especially important on large, complex systems whose requirements may contain several interdependent parts. ("We split a requirement to deal with different aspects... like functionality.") IS-A links between requirements show the parent/child hierarchical relationships. ("Higher levels of requirements are just abstractions of lower levels. Initially its a top-down, hierarchical decomposition starting with the most general goals of the customer, flowing down to a set of systems performance requirements... you now can have children with multiple parents, whereas originally we were probably looking at children with a single parent.") IS-A hierarchies can be used to show the relationship between requirements specified for an entire class of hardware to be used and those for a specific type of hardware component.

5.2 Rationale Submodel

The following is a discussion of the Rationale submodel presented in Fig. 4. The specification, elaboration, decomposition, derivation and modification of OBJECTS such as REQUIREMENTS, DESIGNS, etc. GENERATE (or lead to)

ISSUES or CONFLICTS, often due to the differing interpretations, assumptions, interests, viewpoints, experience, and objectives of the stakeholders. Information about how decisions are made to resolve these ISSUES must be maintained throughout the system lifecycle to ensure that customer requirements are understood and satisfied. (*"Traceability pays for itself by making sure you understand how the requirement is interpreted, how it is being implemented, and what were the issues that were considered in the design to satisfy that."*) These DECISIONS, in turn, may AFFECT REQUIREMENTS.

Various ALTERNATIVES that address the resolution of issues are considered. (*"We want everyone to know what the options are, why we are picking one versus another. They can immediately see if these decisions will affect their parts and this feedback is important in making the right decisions. Getting a resolution through this process is also helpful in bringing everyone on board and on the same sheet of music."*) ARGUMENTS for and against each ALTERNATIVE may be proposed. The DECISION to SELECT one or more ALTERNATIVES is often INFLUENCED by the CSFs. (*"All parties understand the various possibilities and why one looks better than others in terms of what is critical for the project."*)

A common observation made by most focus groups is that such detailed information on all the ALTERNATIVES considered, the ARGUMENTS supporting and opposing these alternatives and the assumptions behind these decisions are seldom explicitly recorded. Most often only the chosen alternative is well-documented and the discarded ones are not recorded. With evolving requirements, availability of such information may help avoid expensive rework. (*"We debated that for two weeks now. Why was it thrown out years ago when we got one camp arguing to put it back in and another to keep it out. And, if we could have gone back and assessed why we made that decision two years ago it would have saved us a lot of time."*)

Often, the capture of rationale at the detail described above is impractical due to overhead involved in capture and due to the lack of tools to facilitate it. However, simple descriptions of rationale on which REQUIREMENTS or DESIGNS are based may be recorded along with the assumptions behind them. Further, ASSUMPTIONS underlying the various components of the deliberation are also recorded. (*"We record the rationale as simple notes, and add specifically any assumptions we have made. It is useful to keep track of these assumptions as clearly as possible for these are the most likely candidates for changed requirements."*) We return to this issue in Section 8.5.

Over the lifecycle of large projects, requirements, assumptions, etc. change. Further, such projects also experience personnel turnover. If a detailed rationale is not maintained, these changes lead to a lot of rework. (*"We already looked at that trade-off, that approach, but the person who did it—the analysis—is leaving and the person that reviewed it is gone, so we churn the whole thing again."*) (*"This is a lengthy look at all these issues with a lot of people that contribute to, touch, change, add, use this information."*)

During the systems development process, different stakeholders often bring different interpretations of requirements and their origins. For example, even when two

different stakeholders review the same origins of a REQUIREMENT (say, a standard) they may interpret it differently. Often the difference in interpretation is the cause of conflicts between stakeholders, even in a cooperative setting. (*"You can look at a given set of requirements and they might be meaningless if you do not really know what the customer needs because you can interpret them wrong, or at least two or three different ways."*)

Another reason to have detailed rationale is to provide accountability. Information about how the requirements and design have evolved, what changes have been made, why and how they were made, and the status of their development is extremely helpful to those stakeholders that were not involved in creating the requirement. (*"If you do not have traceability to the rationale, it is impossible to understand many low level requirements. They may have made sense to the person who evaluated the options and not for others."*)

5.3 Design/Allocation

We use the term design to refer to any activity that creates artifacts, including implementation. In Fig. 5, REQUIREMENTS DRIVE DESIGN, that are often BASED ON MANDATES such as STANDARDS or POLICIES or METHODS that govern the system development activity. (*"We have to follow military standards and policies to make sure we are compliant."*)

SYSTEM/SUBSYSTEM/COMPONENT (hereafter referred to as COMPONENTS) are the building blocks of the system. They are DEFINED or CREATED by the DESIGN process. REQUIREMENTS are ALLOCATED to COMPONENTS that are supposed to satisfy them. The components could be a piece of hardware, software, humanware, etc. (*"At the allocation step you are going to allocate to people, to processors, to memory, to hardware, etc."*)

COMPONENTS DEPEND ON other COMPONENTS in that the functionality and performance of one may depend on another. When the interfaces between different components are explicitly identified (for example, in interface specifications), maintenance of dependency information is easier. The difficulty in the identification of indirect dependencies among COMPONENTS makes it important to record them when they are indeed discovered. For instance, an operating system on one component may affect the choice of hardware on another, even though there may be no direct interfaces between them. When a component is created or modified, this traceability information is helpful in determining the impact on the entire system. (*"This information is hard to figure out and people who know about how your work on a piece affects some seemingly unrelated piece do not share this information, leads to a lot of rework."*)

The composition of COMPONENTS is identified using the PART-OF links. This information allows the designers to identify intercomponent dependencies. (*"As I change a part or break it into subparts, I would like to have some sort of traceability or mesh structure which gives some sort of relationships... I would like to know what is the characteristic of each component."*) The IS-A hierarchy specifies that components are organized at different levels, from system to subsystem to item and to unit. (*"We want to keep track of common details for the same types of components."*)

RESOURCES (such as money, weight, personnel, power, etc.) are USED-BY COMPONENTS. As discussed

in Section 5.1, when these resources are CSFs, it is important to track their allocation, distribution and utilization during design and implementation. ("At the allocation level there needs to be parameters that associate budgets of resources that are going to be consumed... and then you are going to have to test to see that you are somehow compatible with, complying with and not grossly out of bounds with your budgeting.") ("I want traceability to give me some level of assurance of particular steps in the design process, that I am meeting or likely to meet, system goals or to help me to allocate resources or capabilities in such a way that I maximize the likelihood of meeting system goals.") ("If cost happens to be the major system driver, then what I am really interested in is keeping track of actual or supposed cost of the components. If, on the other hand, memory space is the primary driver, they might not really pay as much attention to cost, as how my decisions impact the allocation of memory space.")

Several focus groups mentioned that it was important to identify the *FUNCTIONS PERFORMED BY COMPONENTS*. These *FUNCTIONS* are typically traced to the functional *REQUIREMENTS* explicitly identified in requirements documents. ("...we have to keep the functionals separate. We want to make sure each of these is addressed in some way by the different pieces of the system.") ("Another use of traceability in design is allocating functions in such a way that you are sure that they support each other and do not interfere with each other.") A *COMPONENT* may partially *SATISFY REQUIREMENTS* and, as the design progresses, all *REQUIREMENTS* must be fully satisfied. The difficulty in explicitly creating traceability links between nonfunctional requirements and system components was also highlighted by several groups. ("...with nonfunctionals, it is a problem. How are we going to say which part of the system affects, how much of a performance requirement to give a 20 millisecond response.")

The *COMPONENTS DEPEND ON EXTERNAL SYSTEMS* in that they often interface with them. Direct interfaces are easier to recognize and represent in a traceability framework than indirect dependencies. ("A particular product line is going to be brought on board a ship and its going to have to plug into an already existing system... the things they need to know is what is the functionality that the system needs to have, what is the demand on these services that they would have, and the range of the demand... especially with respect to the system we are plugging into.")

Our discussion in Section 5.2 on the capture of rationale applies also to the capture of rationale behind the development of *DESIGNS* and *COMPONENTS*. ("Traceability would come in handy for determining the reasons for your design.") ("That is the design knowledge capture that we were supposed to maintain. What is basically here is the design the way it is and here is the decisions, the thought that went into that decision. And then when you changed them, here is how we changed them.")

5.4 Compliance Verification Submodel

A variety of CVPs (*PROTOTYPING*, *SIMULATION*, *TESTING*, and *INSPECTION* shown by the *IS-A* links) are developed to ensure that each of the requirements is adequately addressed. ("We write the test specifications by taking apart the *Software Requirements Specifications* and *Component*

Specifications.") ("You have got to be able to specify what you are going to test taking *Standards* into account.") The ability to develop CVPs is a critical factor in deciding whether a requirement is considered as such. ("If something is untestable or we elect not to test, then it becomes no longer a requirement, or at least we all agree, that it is unverifiable.")

The development of CVPs is also governed by their *USE* of *RESOURCES* (such as personnel, money, etc.). The utilization, assignment and availability of resources often determines whether one can even perform a compliance test or if the cost-benefit analysis of performing it justifies it. The trade-off between the resources allocated to development and CVPs is often a source of *ISSUES* or *CONFLICTS* that must be resolved. ("How are we going to find that this is how far we should go in testing. It is a matter of trade-off, and for discussion.") The rationale submodel can be used to represent the process of resolution of these.

MANDATES such as *STANDARDS*, or *POLICIES*, or *METHODS* are commonly the basis of CVPs and determine which CVPs are required and how they are to be performed. Often, requirements are written such that they comply with these. ("Before we decide on a testing strategy, we want to go to the same standard which the requirement is trying to meet.") It is important that the CVPs for a requirement use the same interpretation of the standards used while developing requirements. In the absence of traceability between the *REQUIREMENTS* and the *STANDARDS* on which they are based, it is difficult to develop CVPs. ("There is a lack of a way to firmly assure that the tests you are generating really reflect the same thing as the standard used in writing the requirement.")

CVPs produce results that either *verify* how the *COMPONENTS* satisfy *REQUIREMENTS* or help *GENERATE CHANGE PROPOSALS* for *REQUIREMENTS*, or *DESIGN*, or *IMPLEMENTATION*. They are thus used to certify completeness and correctness of the system and identify changes that may be necessary to meet the objectives. ("As the design or requirement is changed based on tests, my current system continues to support my objectives.")

6 IMPLEMENTING HIGH END TRACEABILITY MODELS: A CASE STUDY

In this section, we illustrate an advanced usage of the models presented in Section 5 with a case study adopted from a system for a U.S. Government organization.

The case study is based on the combat systems section of a new ship (referred to as ECS hereafter).³ The discussion highlights the implementation of traceability at various stages of systems development. We focus on the development of a passive sonar for the ECS, and describe how traceability, using the high-end models described in Section 5, can be implemented in this project. This implementation is described using scenarios at various stages of the development life cycle. The objective of our discussion is to illustrate implementability and tailorability of the models. In this section, the *Requirements Management*, *Design allocation*, and *Rationale management* models are used for this purpose.

3. The discussion has been "sanitized" to protect proprietary or confidential data.

TABLE 5
Traceability Concepts Used in SLATE

Concept	Icon
Traceability Link	
Abstraction Block	
Defining Object	
Complying Object	

The implementation of the models has been made using a popular commercial CASE tool, viz., System Level Automation Tool for Engineers (SLATE). SLATE's traceability scheme is represented using links among abstraction blocks (which are building blocks used to represent physical and conceptual objects). User defined subtypes of the *TRACE LINK* object can be used to represent specific types of links. The link subtype is identified within parentheses with every link. In SLATE, an object that helps define another object is termed a defining object. Objects that comply other objects are called complying objects. For example, a traceability link from a system objective to an organizational need will identify the mission need as defining object. Also, a traceability link from a system objective to a requirement will identify the requirement as a complying object. Table 5 provides the legend for Figs. 7, 8, 9, 10, 11, 12, 13, 14, 15, and 16 containing these traceability concepts used in SLATE.

6.1 Requirements Management

The traceability scheme used for requirements management is adopted from the high-end requirements management model and implemented in SLATE, as shown in Fig. 7. Specifically, it shows the linkages among requirements, mission needs, system objectives, and critical success factors. It also includes components of the rationale model to represent the rationale behind requirements.

Mission Needs. According to this traceability model, a first step in a system development activity is defining the MISSION NEEDS that *JUSTIFY* the SYSTEM OBJECTIVES. The MISSION NEEDS are provided in a document called the MISSION NEEDS STATEMENT (MNS). This document, a segment of which is shown in Fig. 8, resides as a living document in the SLATE database. The MNS document contains the "big picture" requirements for a Surface Combatant ship, the subject of the ECS. It provides general guidelines on the mission, capabilities, design and architecture constraints, and personnel constraints. It also provides some operational constraints.

Each mission needs identified in the MNS is represented in SLATE as an abstraction block. For example, Fig. 9 (line 3) shows that a mission need is represented by the abstraction block no. 7, which is also specified as a defining object. In contrast, line 5 identifies a complying object, viz. the system overview. This, in fact, represents the SYSTEM OBJECTIVES (corresponding to the ORGANIZATIONAL NEEDS in Fig. 3).

The tracing of relevant parts of SYSTEM OBJECTIVES with statements in the MNS is shown in Fig. 7 using the *JUSTIFIES* link. The project sponsors involved in the

definition of SYSTEM OBJECTIVES create these linkages to justify each objective based on their relation to higher-level mission needs, i.e., provide "backward traceability" to the origins of requirements. During discussions about the rationale for different functionalities of the system, this traceability will be valuable. For example, the systems analyst can use this information to initially get the "big picture" of what the customer wants the system to do and why.

Requirement Identification. The next step in the development is the identification of requirements. As shown in Fig. 7, REQUIREMENTS are *GENERATED* from SYSTEM OBJECTIVES. The original requirements document for the ECS that includes requirements for the Passive Sonar Requirements is maintained as a living document in SLATE. A SLATE utility called Requirements Identifier parses documents to identify statements that meet user defined criteria for identifying requirements using keywords. In our case study, in compliance with project standards, sentences that contain the word "shall" are identified as distinct requirements.

Parsed text subsequently becomes REQUIREMENTS OBJECTS. In SLATE terminology, these REQUIREMENTS OBJECTS *COMPLY* with the SYSTEM OBJECTIVES. Fig. 10 shows System Objective (2.0 System Overview) with several complying requirements (Requirements 3-2 through 3-5).

Requirements Rationale. Fig. 11 depicts information about a particular REQUIREMENT 3-3. It identifies the broadband figure of merit at a geometric mean frequency in a radio frequency band against a source level target. Requirement 3-3 has a link to the paragraph in the source document from which it was identified (see Line 5 in Fig. 11). This source paragraph is identified as the defining object. In SLATE, RATIONALE can be attached to REQUIREMENTS in the form of notes. A user defined NOTE type called RATIONALE is used for this purpose. In our example, REQUIREMENT 3-3 is *SUPPORTED BY* a RATIONALE note (see line 7). This RATIONALE addresses the Figure of Merit (FOM) required to achieve sufficient detection range against most likely submarine threat. Requirement 3-3 is also linked to a paragraph from the system objectives that generated it.

6.2 Design Allocation

The high-end design allocation model has been tailored for use in the ECS problem. The model shown in Fig. 12 identifies the linkages between requirements, design, system components, and functions. Further, it identifies resources used by system components as well as standards/policies/methods that constrain the design. In this model, the link between standards/policies/methods and design has been defined as the inverse of the link⁴ shown between these objects in the high-end design allocation model shown in Fig. 5.

During the design phase of ECS, Data Flow Diagrams (DFD) and Entity Relationship Diagrams (ERD) are used to capture the information flow in the system. SLATE provides an interface with TeamWork⁵, a CASE tool used in ECS for this purpose. When abstraction blocks in SLATE are linked with the DFDs in TeamWork, objects called

4. As mentioned earlier, for each link in our models an inverse may be defined.

5. Trademark of CADRE Technologies.

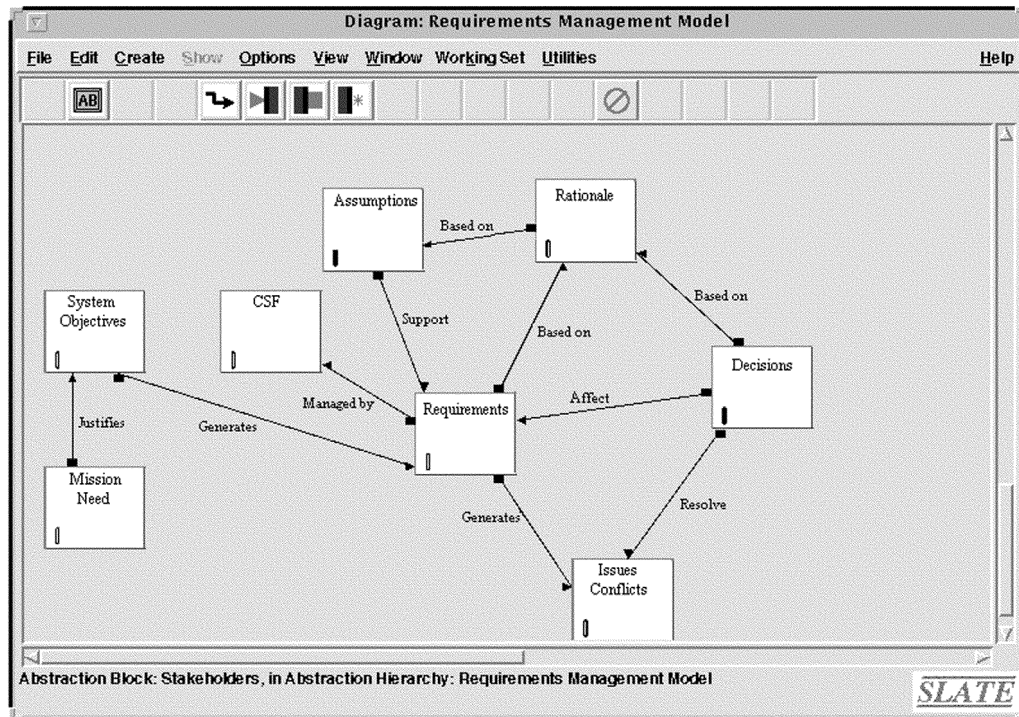


Fig. 7. Requirements management model.

TeamWork handles are formed. Fig. 13 illustrates abstraction blocks representing the sonar and its components, as well as the TeamWork handles for the corresponding DFDs. Here the TeamWork handles represent the traceability link between information represented in SLATE and TeamWork. The SLATE objects are treated by TeamWork as requirements attached to individual processes in a DFD.

SLATE provides a mechanism for synchronizing TeamWork when objects change in SLATE. A similar mechanism can be used to link requirements represented in SLATE and functions represented in TeamWork.

Fig. 14 shows the defining objects for Requirement 3-3, namely its source document paragraph, its rationale, and system objectives. The first complying object (line 6) is a

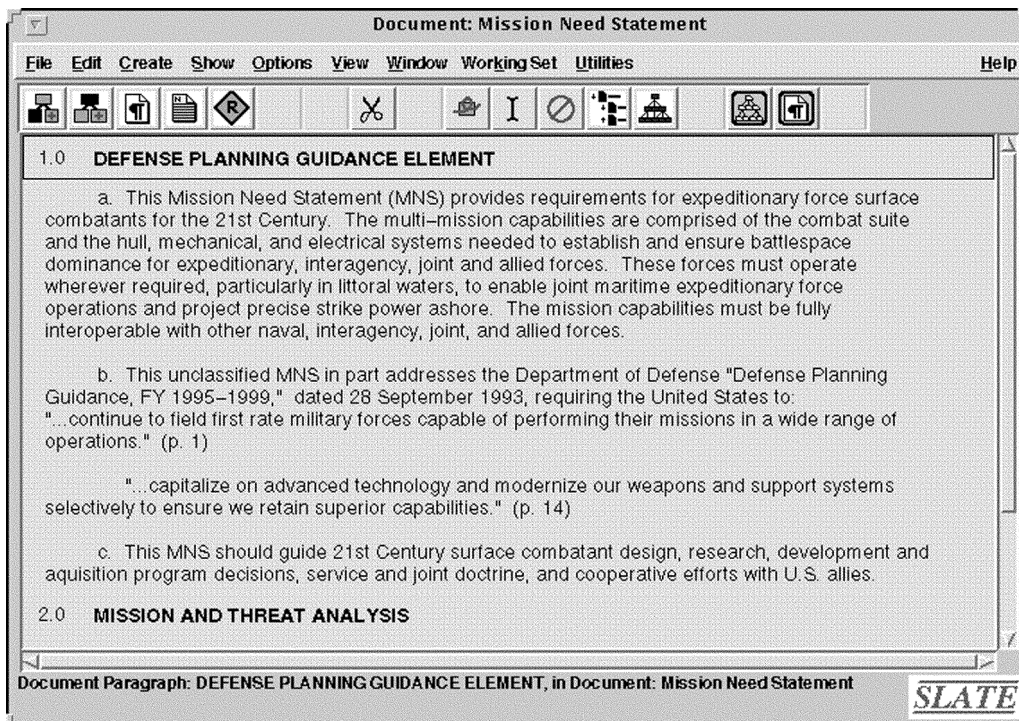


Fig. 8. MNS document.

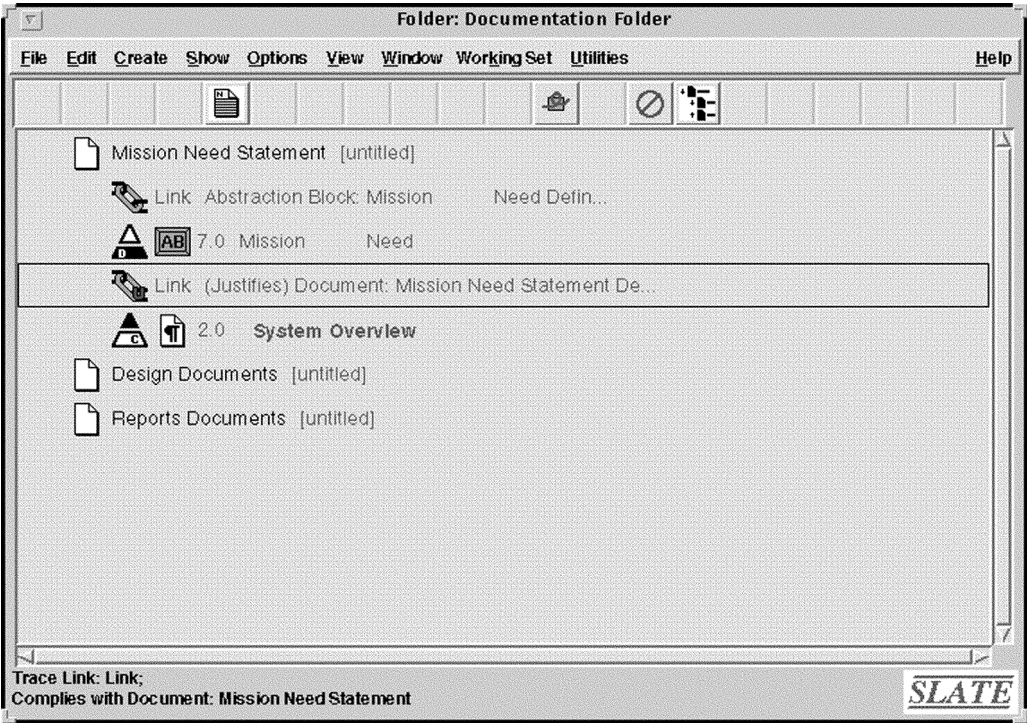


Fig. 9. Traceability links.

cell (Row 10, column 3) in a table that contains the constraints that must be satisfied by the design. These constraints were created by the design team based on the analysis of requirements. In fact, the value in this cell references a string in the requirements statement. Now, any changes made to the requirement will be automatically reflected in the constraint. In our example, the requirements statement mentions the need for BF

microcode which, in turn, is referenced in the cell. If the requirement 3-3 is changed from BF microcode to SC microcode, it will automatically change the constraint on the functional design. Requirements in this project often contain such parameters that may get modified during development. Maintaining information in one location and referencing it in REQUIREMENTS (as well as other relevant objects) significantly increases the integrity of the

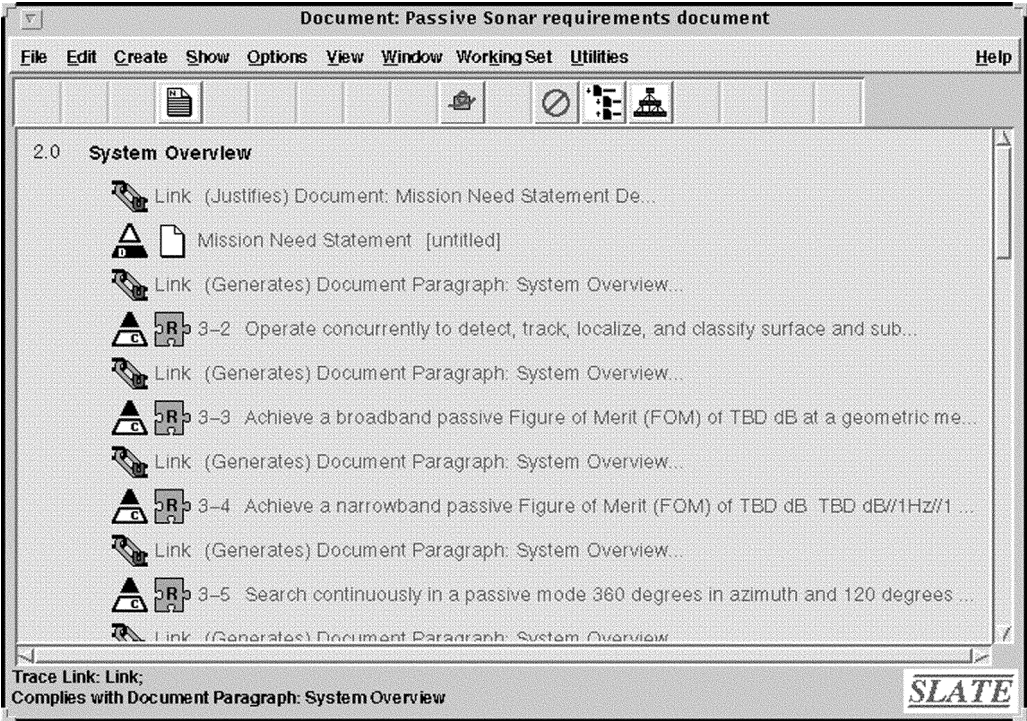


Fig. 10. Requirements compliance with system objectives.

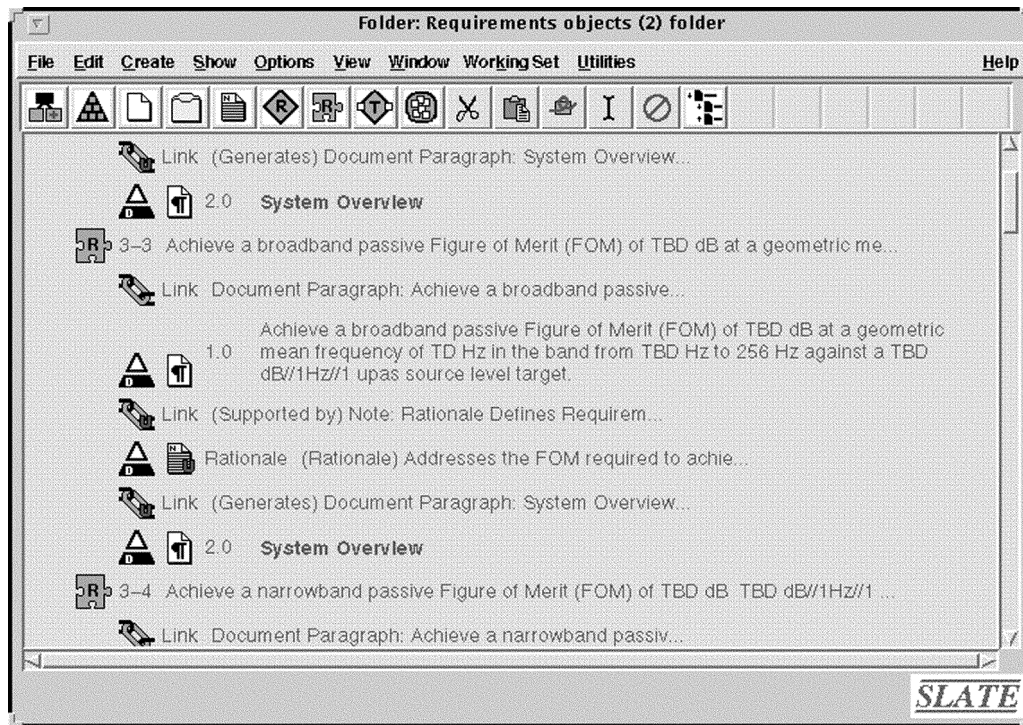


Fig. 11. Requirements rationale.

information besides reducing the possibilities of overlooking important changes.

The next two objects shown in Fig. 14 are two functions represented as abstraction blocks, viz. 1.2.1.1 Signal Conditioner and 1.2.1.1.1 Digital Beamformer. The

Requirement 3-3 has been allocated to these blocks that depict the functional view of the passive sonar system. These could just as easily describe any other view, for instance, the physical system components to which the requirements are allocated.

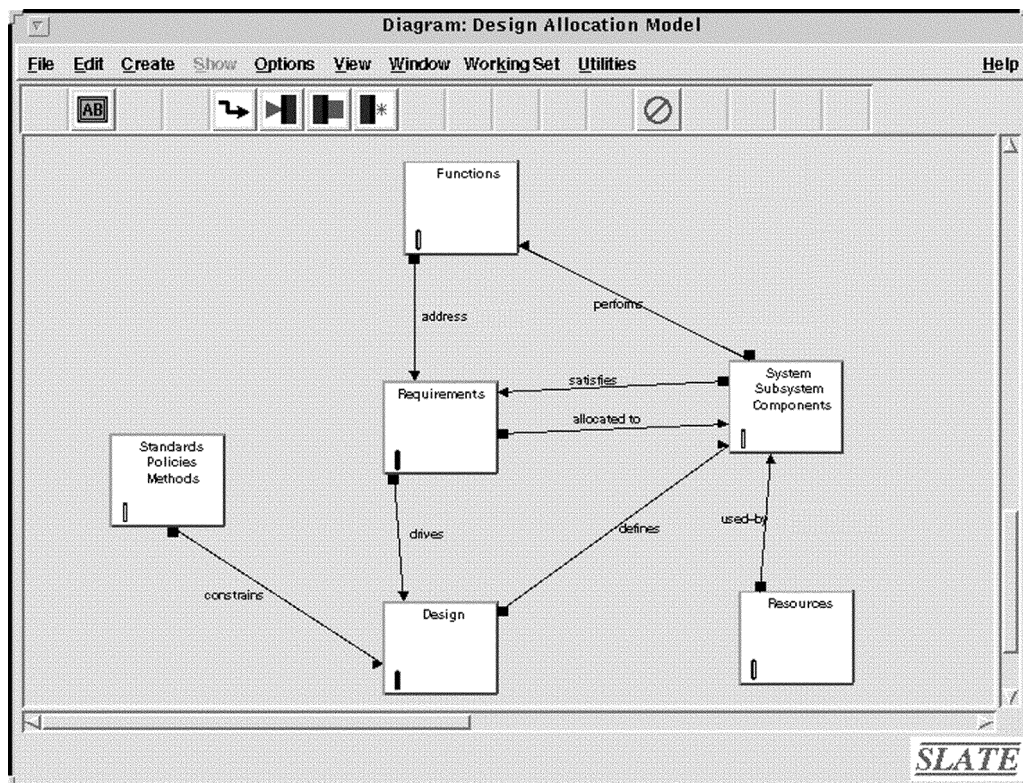


Fig. 12. Design allocation model.

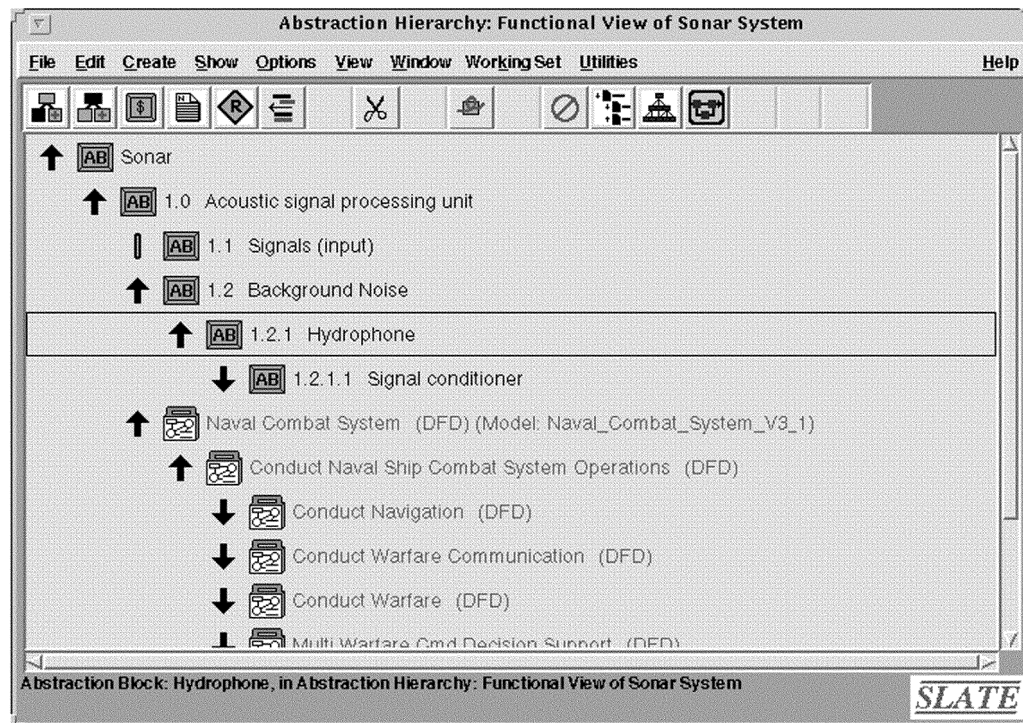


Fig. 13. Design linked to system components.

In SLATE, CRITICAL SUCCESS FACTORS (CSF), such as resources, can be modeled using budgets. In ECS, REQUIREMENTS are managed by CSFs (as shown in Fig. 7) and resources are used by system components. These are represented by attaching budgets to relevant objects such as REQUIREMENTS and SYSTEM COMPONENTS. Fig. 15 shows that Requirement 3-9 is linked to a component

(abstraction block 1.2.1 Hydrophone) to which a budget for hull size is attached. Budgets are used for planning as well as for performing “what if” analyses. SLATE supports hierarchical budgets (similar to nested spreadsheets). For example, the budget for the hull size attached to the Hydrophone can be linked hierarchically to the budgets for its children and so on. Any allocations done at the lowest

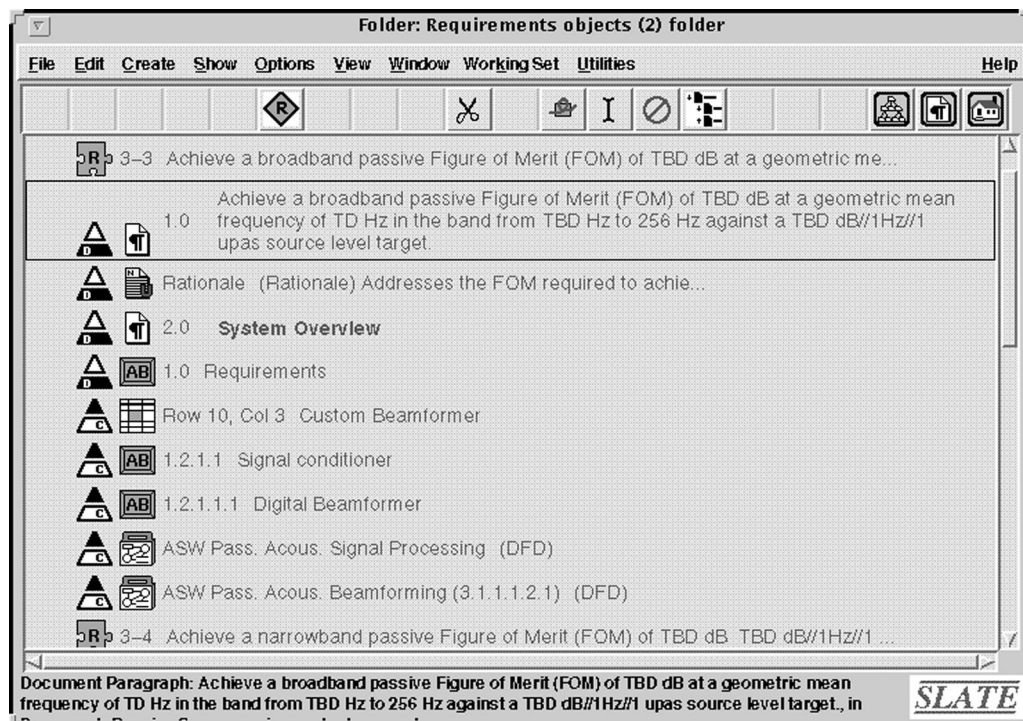


Fig. 14. Traceability between requirements and design.

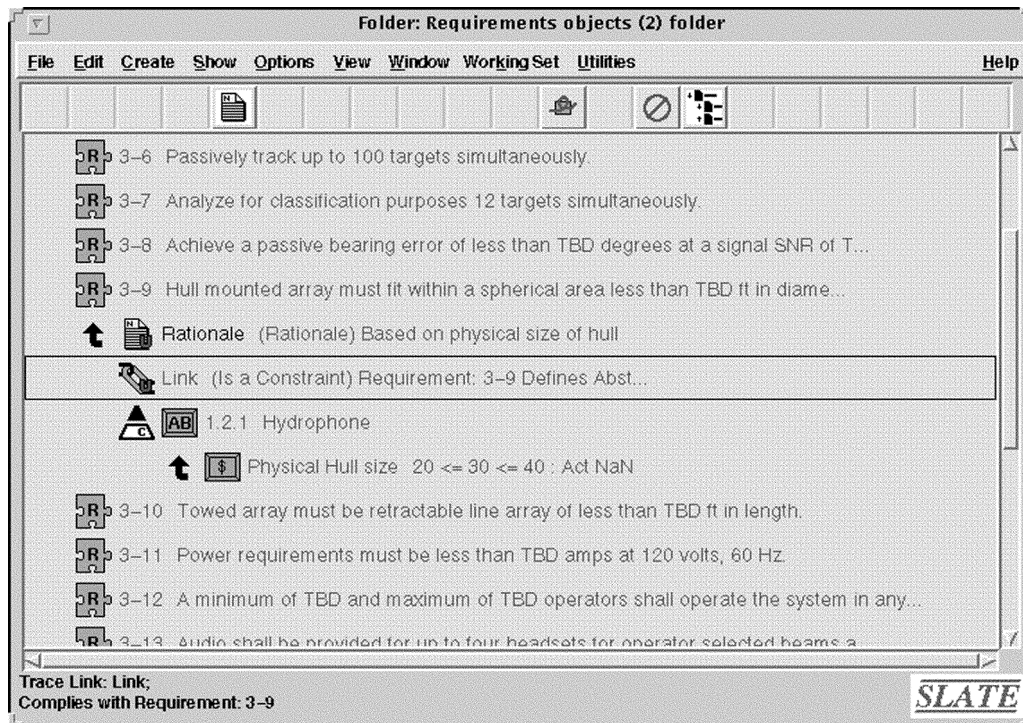


Fig. 15. Budgets for modeling resources, critical success factors.

level (leaf nodes) can be “rolled up” the hierarchy. Another potential use for budgets in complex projects such as ECS is the ability to monitor the cost and schedule of a project.

6.3 Rationale Management

Discussions about requirements may raise issues or conflicts. Alternative ways of addressing these issues are explored and decisions to resolve those issues are made. ISSUES, ALTERNATIVES, ARGUMENTS, and RATIONALE

(which are used to represent rationale, as shown in Fig. 7) can be represented in SLATE using different types of notes.

Fig. 16 shows an example. The ISSUE relates to the REQUIREMENT of tracking 100 targets. Two ALTERNATIVES 1 and 2 are proposed. Alternative 1 suggests that the system should “keep the capabilities of simultaneously tracking 100 targets.” Alternative 2 advocates that the design “reduce the capabilities of simultaneous tracking to 50 targets.” Argument 1 supports Alternative 1

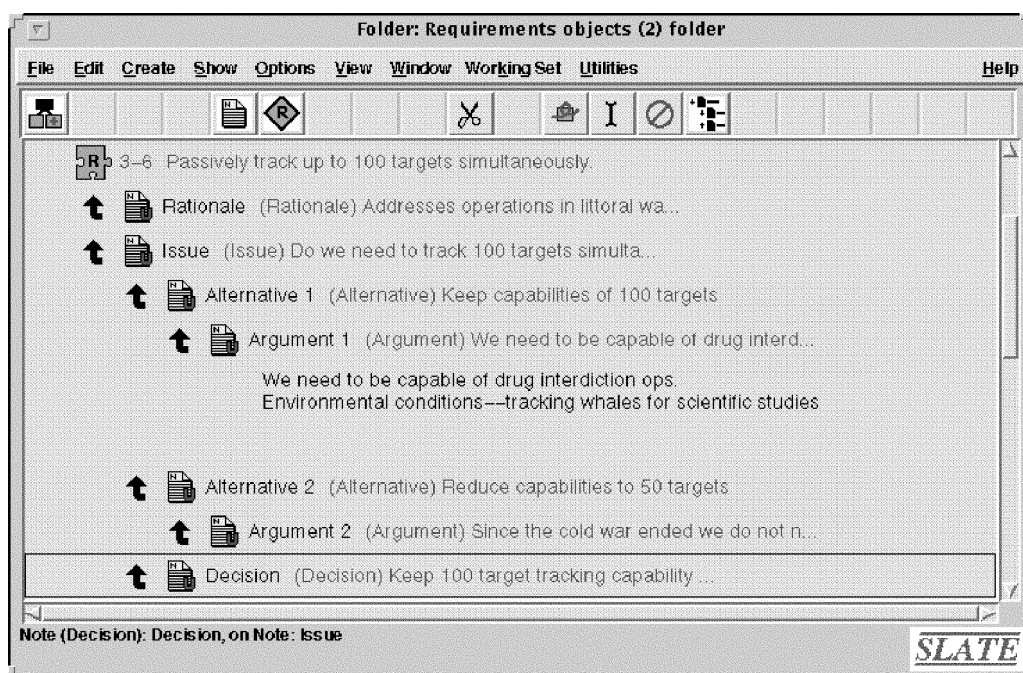


Fig. 16. Rationale capture.

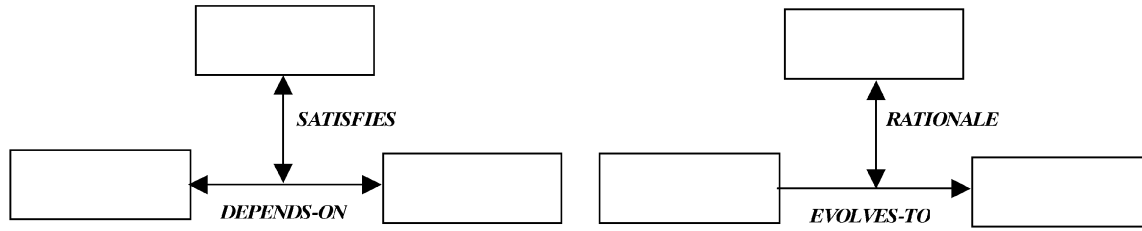


Fig. 17. Product and process-related categories of traceability links.

by stating that “we need to be capable of drug interdiction operations...” Argument 2 supports Alternative 2 by stating “since the cold war ended, we do not need the capability of tracking 100 targets simultaneously.” Based on these ARGUMENTS, a DECISION is made to keep the capability of tracking 100 targets simultaneously.

Each NOTE type can be assigned attributes. For example, an attribute to identify the “originator” of the ISSUES and ARGUMENTS can be used. Capturing this type of information is useful when a project is completed and the customer questions why something was implemented a particular way. Further, this information is useful while performing system maintenance. Capturing the original issues and decisions can prevent rework on the same issues that have been resolved long ago. The mechanism described here can be used for maintaining rationale in any phase of the lifecycle.

In this section, we have illustrated how the high-end models proposed in Section 5 could be tailored and implemented in large-scale system design situations. This also provides a concrete example illustrating the instantiation of the model components. By using a commercial CASE tool, we further highlight the implementability of the reference models. In several such case studies, the models proposed in Section 5 have been shown to be both adaptable and implementable within currently available commercial environments.

7 SUPPORTING THE TRACEABILITY LINKS

Summarizing the differences between the low-end and the high-end uses of traceability, we observe that the latter are characterized mostly by two extensions:

- They employ a much richer type system for the product-related object and link categories, thus enabling easier retrieval and more precise (human or computerized) reasoning about traces.
- They have a much stronger emphasis on the process-related categories; indeed, in the long-term case study conducted as part of this work [10], the step from low-end to high-end traceability was followed by a strong increase in the SEI capability maturity rating.

The large number of different link types precludes an individual treatment of each one. However, as shown in Section 7.1, we can group the link types found in the reference models into a few basic categories. We then go again back to the empirical studies in order to identify the

necessary tool support for each category, and point out proposals in the commercial or research literature that may address these issues. Conversely, this discussion then also provides a classification of the solution proposals advanced for traceability tool services.

7.1 Basic Classification of Traceability Links

Formally speaking, a traceability system can be defined as a semantic network in which nodes represent objects (also stakeholders and sources which we do not consider at the moment) among which traceability is established through links of different types and strengths. This dependency-directed approach of maintaining consistency of designs dates back at least to the work of Stallman and Sussman [41] and is by now well-established.

The simplest traceability tools are purely relational (i.e., in the form of relational databases or spreadsheets) and do not systematically distinguish different node and link types. Others, such as RDD-100, use a basic entity-relationship structure to distinguish nodes and links and allow the user to introduce distinctions between different types of nodes and links. However, this begs the question of which node and link types *should* be defined.

In the literature on software information systems, a large number of link types have been proposed, many of which are also important for traceability. The basic organization and abstraction mechanisms, independent of traceability, are detailed in [39]. In the NATURE project, a comprehensive literature survey revealed 18 different types of dependencies in the traceability context [11]; these were grouped into five clusters named *condition*, *contents*, *documentation*, *evolutionary*, and the already-mentioned *abstractions*.

As a starting point for classifying our empirical observations, we adopt a simple system of just four types of traceability links, shown in Fig. 17. A first group of traceability links and mechanisms could be called *product-related* because they describe properties and relationships of design objects independent of how they were created. In Fig. 17a, the high-level object (e.g., a requirement, a standard, a policy, or complex design object) defines some kind of constraint or goal which should be *SATISFIED* by one or more lower-level objects. Satisfaction is usually just a *claim* that must be substantiated by compliance verification procedures (cf. Section 5). The shared constraint or goal to be satisfied also implies a *DEPENDENCY* between lower-level objects. Generalization and aggregation abstractions (i.e., configuration of complex objects) are special kinds of dependencies. Thus, we have two basic kinds of product-related traceability links, satisfaction links and dependency

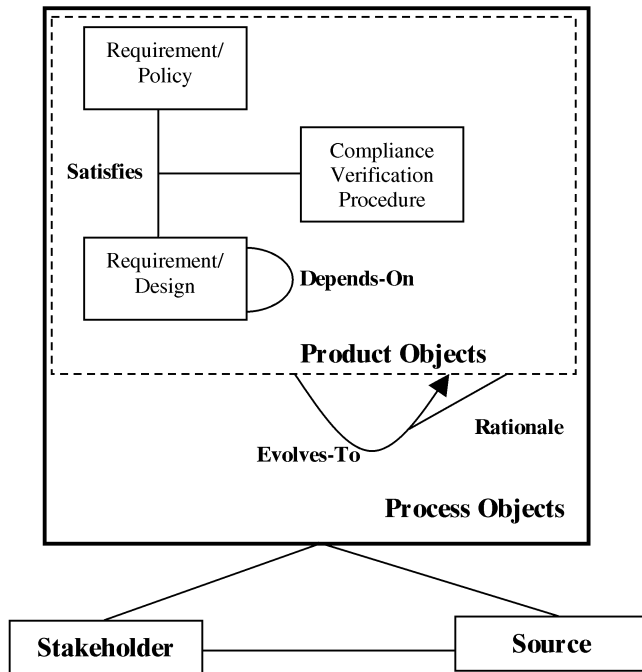


Fig. 18. Traceability meta model extended with principal link types.

links; low-end traceability users tend to be characterized by relying mostly on these two link types.

The second group of traceability links is called *process-related* because it can be captured only by looking at the history of actions taken in the process itself and cannot be recovered from the product relationships alone. Fig. 17b looks very similar to Fig. 17a, with the important difference that the evolution link types have a *temporal direction*: The left lower-level design object *EVOLVES-TO* the right one through some kind of action whose *RATIONALE* is captured in the higher-level object. Thus, the two kinds of process-related links are evolution and rationale links. Advanced traceability users typically have a higher share of link types belonging to this category.

If we enhance the traceability meta model of Fig. 1 with these four traceability link types, we get an “onion-shell” meta model of traceability, as shown in Fig. 18. Reflecting back to our empirical studies, the models used by low-end traceability users tend to focus on the inner kernel of the model more advanced users tend to venture further out to process-related issue and more explicit source and stakeholder management.

As evidence for these observations, Table 6 summarizes the link types of the high-end models according to how and where they are used and the link category they belong to. The abbreviations in the right column refer to the sub-models in Section 5 where the link types occur: RM for Requirements Management, R for Rationale, DA for Design/Allocation, and CV for Compliance Verification.

7.2 Issues in Implementing Traceability Link Types

Besides the classification of the different link types, a second goal of our study has been to identify issues in implementing their capture and use in system engineering practice. For each of the four link types discussed in the

previous subsection, we first present the issues raised by our empirical study participants and then discuss solution strategies proposed in the literature. The overall result is that local solutions exist for all the identified issues; however, the question of how to configure such individual solutions to usable and more powerful trace tools than those on the market today remains open. In Section 8, some more general design goals for such systems will be discussed, and an example of an interesting solution will be given where several of the issues addressed in this section have been combined in a creative manner.

7.2.1 SATISFIES Links

The focus groups raised three main issues concerning support for the satisfaction links:

Derived Links. An important use of requirements traceability is to ensure that the system meets the current set of user requirements. Representing this information is considered to be critical from a project management perspective. It is usually accomplished by creating a link between the set of requirements and a set of system components that *SATISFY* them. Such links may also be indirectly derived. An important concern of the study participants was the lack of support in many CASE tools for the automated identification of derived links. (“I do not have the time to link every requirement with everything produced at different stages. These links must be automatically derived whenever possible.”) For example, requirements may be linked to designs that are intended to satisfy them. Designs, in turn, may be linked to relevant system components. Then, a derived link is created from requirements to system components.

Mechanisms for automated inferencing incorporated in deductive or active database environments such as ConceptBase [35] can be used to infer “derived links” in a traceability environment. As a critical first step, the semantics of the different linkages among objects must be clearly represented.

Multiple Sources of Support. When a set of requirements may be satisfied by a set of system components, the status of a requirement (whether satisfied or not) can be ascertained based on whether the relevant set of system components meet their goals. Most tools do not provide a way to represent the fact that alternate ways may exist for satisfying a requirement. (“We cannot state that the requirement is met by either this solution, or another one, or a combination of system components easily in these tools. This will give me the flexibility to accurately convey the multiple solution paths I need to follow anyway.”)

A goal tree is a good representation for answering questions about how goals were achieved and why they were attempted [42]. Well-defined inference procedures for finding “optimal” solutions from a goal tree can be used in finding the best possible way to satisfy a requirement or resolve an issue (say, with least cost) if they are represented as a goal tree. In the context of system engineering, the goals could correspond to requirements that need to be satisfied, issues that need to be resolved or complex decisions that need to be made. Each of these “objects” in our model can be represented by a goal tree. The nodes of a goal tree are of two types: OR nodes represent choices, i.e., they are satisfied when

TABLE 6
Traceability Link Types Used in Reference Models

LINK TYPE	PURPOSE	USES	EXAMPLES MODEL : LINK
Satisfaction Links	To ensure that the requirements are satisfied by the system. Relationships between one or more design, implementation objects and requirement objects verified by CVPs	to ensure consistency between outputs of different phases of the life cycle track the designs created to satisfy requirements track system/subsystem/ components to which requirements are allocated identify the system/subsystem/ components that satisfy requirements and that every component satisfies some requirement track CVPs that are developed for each requirement track the results of CVPs to ensure that the requirements are indeed satisfied	DA : requirements-drive-design DA: requirements-allocated-components DA: components-satisfy-requirements DA: systems-perform-functions DA: functions-address-requirements CV: CVP- developed for – requirement CV: (system - satisfies - requirements) - verified by CUP
Evolution Links	Document the input-output relationships of actions leading from existing object(s) to new or modified object(s)	Identify where do the various objects come from. I.e., identify the origins of various objects to facilitate: better understanding of requirements (and other objects) establishment of accountability for creation and modifications of objects tracking the modification, refinement history of various objects.	RM: system objectives-generate-Requirements; RM: Organizational needs - justify-system objectives RM: scenarios-describe-System objectives Requirements} RM: Change proposals-modify-requirements RM: system objectives-generate-change proposals RM: organizational needs-identify-CSF RM: requirements-based on-Mandates RM: requirements -derive-requirements RM: requirements-elaborate-requirements CV: CVP-generate-change proposals CV: CVP-based on-Mandates; R: decisions-affect-Requirements DA: requirements-drive-design DA: design-based-on-Mandates DA: change proposals-modify-design DA: designs-define/create-components

any of the immediate subgoals are satisfied; AND nodes represent simultaneous subgoals that must have compatible solutions, i.e., they are satisfied only when all subgoals are satisfied [43].

Degrees of satisfaction. During systems development, project managers would like to ascertain how close they are to satisfying critical requirements. In complex systems, many requirements (especially, nonfunctional

TABLE 6 (continued)
Traceability Link Types Used in Reference Models

LINK TYPE	PURPOSE	USES	EXAMPLES MODEL : LINK
Rationale Links	Represent the rationale behind objects or document the reasons for evolutionary steps	identify the reasons behind the creation of various object to clearly identify justifications for the creation / modification of objects, important decisions and assumptions made identify the context in which objects are created provide transparency into the decision process including discarded alternatives, in order to - provide clear understanding of the current solution, facilitate maintenance and reuse manage the system development with in line with organizational needs / objectives	R: objects-based on-rationale R: objects-generate-issues_or_conflicts; R: Alternatives-address-issues_or_conflicts R: arguments-support/oppose-alternatives R: arguments-depend on-assumptions R: {Objects Rationale Decisions Arguments}- depend on-assumptions R: decisions-select/evaluate-alternatives R: decisions-resolve-issues/conflicts R: CSF-influence-decisions RM: Requirements-managed by-CSF DV: Components - use - resources
Dependency Link	help manage dependencies among objects (typically at the same stage of development), often imposed by a constraint (resource, competence, compatibility)	track the composition and hierarchies of objects manage the repercussions of changes in one on other objects that depend on it	RM: constraint-ISA-requirement RM: requirement-part of – requirement RM: {strategic need operational need}-ISA- organizational need RM: Resource-ISA-CSF RM: requirements-depend on-requirements CV: {Test Inspection Simulation Prototype}-ISA-CVP DA: components- part of – components DA: components-depend on-external systems DA: components-depend on-components R: {Objects Rationale Decisions Arguments}- depend on-assumptions

requirements) cannot be said to have been “satisfied” absolutely. Often, a designer is interested in maximizing the degree to which a requirement is satisfied. For instance, a performance requirement to provide response time of 100 milliseconds in a missile system may be thought to be reasonably well satisfied if the system meets the requirement within a few milliseconds of this target. Here, 105 millisecond and 95 millisecond response times may be considered to satisfy the requirement with different degrees. Most current tools do not provide natural ways to represent such information. (*“For compliance, I want to track how well I am doing on a requirement based on progress in implementation. It is hard to get this*

information into and out of my tools.’ I need this information for trade-offs.”)

Mylopoulos et al. [44] suggest that goal satisficing (rather than satisfying) may be a more appropriate way of characterizing this situation. The concept of satisficing, as defined by Simon [45], refers to methods that look for satisfactory rather than optimal solutions. Using this framework, a system component may be thought of as contributing positively or negatively towards satisficing a requirement. Hauser and Clausing [46], in the House of Quality, use four categories to relate how each development characteristic affects the customer quality requirement: strong positive, medium positive, medium negative,

and strong negative. Here, we have a measure of not only the directionality (positive or negative) of a relationship, but also its strength (high, medium, etc.). This scheme can be incorporated in our traceability model as follows: A system component may contribute toward satisficing a requirement along these four (or a finer grained) categories. For example, a user interface component may have a strong negative impact on a performance requirement.

All three issues—derived links, multiple support, and degrees of satisfaction—can also occur in combined form. When a requirement is satisfied by a set of component, mechanisms for combining the “evidence” for its satisfaction must be developed. For example, if all the “evidence” is in the same direction (positive), the degree to which the requirement is satisfied may be treated as the minimum of the support it receives. Situations in which a requirement receives both positive and negative support at varying degrees pose a significant problem. Requirements that rely on user perceptions (such as usability of a system) require qualitative measures of ascertaining compliance as well as qualitative mechanisms for combining evidence. Combining beliefs in evidence and propagating such beliefs in such an inference network may be supported with mechanisms such as Dempster-Shafer, Bayesian Probability, Belief Networks, and Fuzzy Logic [47]. Whereas Fuzzy logic provides good mechanisms for representing and combining imprecise, qualitative evidence, the other, more quantitative, methods provide stronger methods for combining evidence from multiple sources.

7.2.2 DEPENDENCY Links

The dependencies created between objects when trying to satisfy some goals are at least minimally supported by almost all traceability models. Issues raised by the study participants therefore address the lack of precision in characterizing the types and strength of such dependencies, resulting in much less automated support than would be possible.

Types of dependency and inferencing services. Most traceability models explicitly represent dependencies among objects. For example, dependencies among system components, requirements, assumptions, and decisions are part of a high-end traceability scheme. The type of dependencies that exist among objects vary widely. For example, a software requirement may depend on a requirement for a specific hardware, implying that in order to meet one the other has to be met. Such a dependency is very different from the relationship between two outputs that must be produced in a predetermined order. For example, an organization may have a policy that the test plans for a requirement must be developed before design solutions that satisfy the requirement are considered. Our empirical studies identified two problems related to the management of dependency links. First, most traceability frameworks simply identify that there exists a dependency between a set of related objects without the ability to specify the nature of the dependency. (“We create matrices that show that there is the hardware requirements on one column that affect the software requirements on the other. This obviously is not good enough. It is not clear whether these two are related as they

consume the same resource like power or because one has to be completed before the other... One needs to intuitively figure out what the connection is.”) Second, as the nature of the dependency is not well specified, the ability to support the use of traceability information is limited. (“We got this dependency as the two components will have to share the same memory. As I keep developing one, the impact on the other must be easy to figure out... how much the total memory is already accounted for. But, my trace matrix stops with just listing the two involved components. There is no help for assessing how changing one affects another.”) Thus, our study suggests that the ability to define different types of dependencies within a traceability framework and developing support mechanisms to manage such dependencies will be very valuable.

In addition to research in theoretical computer science and in operations research, several schemes for identifying different types of trace-oriented dependencies have been proposed in the literature. For example, Yu and Mylopoulos [7] classify dependencies among stakeholders into three categories: goal dependency, task dependency, and resource dependency. This scheme can be adapted and extended to manage dependencies as follows:

- Dependency links among requirements may often be *goal dependencies*. For instance, a software requirement may depend on a requirement for a specific hardware. When a goal dependency between two such requirements exists, inference procedures that monitor whether (or how well) the dependent requirement is satisfied are necessary.
- The dependency between objects created by their common reliance on a task *creates a task dependency*. Design objects (say, the ERD and DFD) often have *task dependencies*. Here, the corresponding artifacts must be produced following a specified method (e.g., structured methodology). If the object (say, ERD) is produced using any other procedure, the correspondence between the two components may not be as expected. When a task dependency exists between two objects, procedures for monitoring the progress of the task using the specified procedure are essential. For instance, whenever a change in one of the artifacts is initiated, all associated task dependencies must be checked for compatibility.
- A *resource dependency* may involve resources that are physical (e.g., money, electricity) or informational (e.g., data). A navigation control system may resource-depend on radar input. When such a dependency exists, the quality, quantity, and frequency of such dependence and the interfaces for the transfer of resources must be identified. When a resource dependency exists, procedures for monitoring that the dependee provides the desired quality and quantity of resources at specified frequency are necessary. It may be necessary to identify who is responsible for the resource and the conditions on which the resource delivery depends.
- Additionally, a *temporal dependency* exists when actions relating to objects are done in a specified temporal order. The temporal order could be specified by operators such as before, after, during,

concurrently, etc. Often system development activities involve strict temporal ordering. For instance, a temporal dependency link could be set up to enforce the policy that test plans must be created before designs for addressing a requirement are completed. Violations of temporal dependency constraints can be monitored using a temporal reasoning system.

Strength of dependency. Another problem associated with maintaining dependency information is that all dependencies are treated as equally critical. Our studies identified the inability to precisely identify the strength of a dependency link to be an important issue for the users of this traceability information. (*"We got two requirements in this matrix, all I know is that changing one will affect the other. In some cases, I really need to take it seriously. The impact can be devastating. But often times it is not that serious. But, I got to ask around to see which ones I should really worry about. It is not in the matrix."*)

Hauser and Clausen [46] propose a scheme for assigning qualitative or quantitative weights to links. Yu and Mylopoulos [7] adapt this scheme in their nonfunctional requirements framework. Based on these, the degree to which a dependency is important to those involved in a dependency linkage is termed the *strength of the dependency*. This is a measure of how much one object affects the other. It could be measured qualitatively (e.g., high, medium, low, etc.) or quantitatively (e.g., 1-10 on a 10 point scale). The strength of the dependency will also very much influence the type of inference procedures and monitoring necessary to ensure the enforcement of various types of dependencies.

A network of dependencies such as those discussed in this section can be defined at different levels of formality [41]. At the most basic level, a link simply identifies the existence of a dependency between two objects. As the only information contained in such a representation is that one object has something to do with the other, the potential for automated reasoning is limited, even when the type of dependency is identified. For example, if there exists a resource dependency between, say, two components in a satellite, we can infer that whenever the weight of one changes, it will affect the other component. A more detailed representation will also capture the direction in which the dependency affects an object. In this example, information on whether the effect of the change is positive or negative can be captured. Finally, a very detailed model will represent the exact nature of the influence that one component has on the other, possibly expressed in a detailed, domain-specific mathematical model. It may, then, be possible to compute the effect of change in the weight of one component on the other. Obviously, the more well-defined the dependencies are, the more sophisticated the inferencing capabilities.

7.2.3 EVOLVES-TO Links

Traceability models represent the process of modification, refinement, and evolution of different objects through a variety of links. The objective is to capture the history of the development in a structured manner so that it will facilitate understanding and management of the objects. For example, in our high-end models, requirements

evolution is modeled using links such as *MODIFY*, *DEFINE*, *ELABORATE*, *DERIVE*, etc. These links can be considered to be specializations of some generic *EVOLVES-TO* link. Our empirical studies highlighted two dilemmas in the management of links representing evolution. First, many traceability frameworks simply represent the fact that an object has evolved, without any elaboration. For example, derived requirements may be simply identified as a new requirement. (*"...do not know where the derived requirements came from."*) On the other hand, very fine-grained representation of the evolutionary path may not be desired in cases where the process is implicitly well-understood. (*"We manage derived requirements in a separate document. As long as we create a simple trace matrix showing the customer requirements and the derived requirements we are fine."*) Another area of concern is the weak support for versioning and configuration management in traceability tools. (*"I have a good sense of how my source code is managed within a configuration system with nice features for notification of changes, but not with my traceability tool."*)

When a fine grained representation of the evolutionary path of an object is not desired, a higher level view can be captured by using just the generic link. Much of the prerequisites traceability information can be thought of as representing the evolution of needs and goals into requirements and, therefore, can be represented using these links. The evolutionary links can be used for chronological replay of the evolution of an artifact. For subsequent phases of development, transformational approaches to software engineering, constraint-based techniques, as well as research in version and configuration management, have created elaborate refinements (and associated reasoning schemes) of the simple *EVOLUTION* link. We do not pursue these specializations further in this paper, but just mention that they can be easily embedded in the framework presented here.

Software configuration management [6] tools provide facilities for representing and enacting policies regarding the evolution of coarse-grained artifacts, and provide mechanisms for notification of the user about changes. These functionalities are likely to be very useful for managing traceability information. However, traceability tools often manage finer-grained objects with more representational diversity, so the combination of configuration management and traceability remains a partially open issue.

7.2.4 RATIONALE Links

Issues concerning the rationale links partially repeat the demand for multigranular representations already discussed for other link types; however, in addition, the capture of the stakeholder dimension of traceability is considered a key issue.

Rationale at different levels of detail. Study participants wish to maintain rationale for different types of artifacts and decisions at different levels of detail. The criticality or the importance of a requirement, for instance, will be a principal factor in deciding the granularity of rationale captured. In contrast, many traceability schemes we observed in practice prescribe a single framework for

representing rationale which is either too fine-grained to justify the overhead in all cases or too coarse to be of value in critical instances. (*"We are supposed to be documenting critical decisions. We are just now going through a review of a very, very critical decision—worth several hundred million dollars. We can not for the life of the project figure out how we came to the conclusion we came to in the first place. It looks doubly bad with the sponsors that we come up with a different answer now. It is all in the details and assumptions we made. But this is where good traceability could have paid off big time... But we cannot do it in this much detail with every decision. We just cannot afford the time for so much detailed documentation for every case... How do we do it right... it is a major issue—we got all these details. As a manager, I cannot get a quick glimpse of the big picture from their reports easily."*)

Most tools for the management of rationale, such as gIBIS [48] and SYBYL [49], as well as traceability tools such as RDD-100, support a uniform approach to the representation of rationale. This results in either a very simple scheme that is not detailed enough to document critical aspects of systems development or a comprehensive scheme too complex to be implemented in every stage of systems development. Also, a detailed study [10] reveals that, when the amount of rationale captured is left to the discretion of the project participants, the quality and content of this information fluctuates depending on time pressure and personal preferences. The rationale management approach taken in the KBSA-ADM tool (discussed in Section 8.4) illustrates a compromise. This tool supports the capture of rationale at multiple levels of detail and also provides a way to map the rationale captured with a comprehensive rationale into a simple view to provide a *"big picture."*

Link to sources and stakeholders. Rationale links help represent the context in which various objects are developed. Our empirical study highlighted two common concerns: The need to link to the various sources of this information to not only reduce the overhead in capturing rationale, but also to ensure minimal loss of detail. (*"As long as I know where to look if I want to understand why we decided the way we did, it is sufficient. I am not only too busy to reenter into the trace tool, I also do not want to interrupt the critical activities to do it."*)

In large projects, the overhead involved in the detailed representation of rationale can be considerable. Facilities to link any object (say, a requirement or a decision) to its sources, such as documents, meeting minutes, e-mail exchanges, etc. will help minimize the overhead. Also, this may facilitate nonintrusive capture. The use of multiple media to capture such informal knowledge has long been recognized in the literature [50], [51] and confirmed in studies of design situations e.g., [52]. The use of thick descriptions for representing traceability is discussed in detail in Section 8.3.

8 IMPLICATIONS FOR TRACEABILITY MECHANISMS

Traceability mechanisms support the capture and usage of trace data, i.e., to document, parse, organize, edit, interlink, change, and manage requirements and the traceability links between them. Besides generic tools such as word processors, spreadsheets, hypertext editors, and relational

databases, a number of specialized RT tools are being offered, either stand-alone (e.g., RDD-100 [53], DOORS [54], SLATE [55]), or embedded in an integrated CASE environment (e.g., Teamwork/RqT [56]).

The majority of present traceability tools offer tabular representations of dependency structures in a relational database formalism. Thus, they are well-suited to support the low-end user who requires little differentiation between link types and uses only very simple allocation and dependency analysis. A few tools allow the user to specialize link types, but it is hard to attach semantics to them. Others offer a fixed high-end set of link types with hardcoded semantics, but tend to force the user to actually supply this detailed information in all cases, even if a simpler model would suffice. Moreover, most tools just offer mechanisms for persistent storage and display of traceability information, but they do not support the *process* of capturing and reusing traces by guidance or enforcement in a systematic manner; those that do tend to have very rigid process models.

Not surprisingly, our empirical studies showed that users are not very happy with the current mechanisms supporting traceability. Some high-end development teams have built elaborate in-house extensions and work-arounds to existing tools which, however, also tend to become inflexible and difficult to maintain. Even high-end users do not always use the sophisticated models presented in Section 5. Instead, as indicated when discussing the various link types, they carefully consider the trade-off between the effort needed to capture complex traceability information and the subsequent value of having this information in each situation in the development process. These observations indicate that a new generation of traceability mechanisms will be needed which we shall try to characterize in this section.

The key observation is the *situated nature of traceability capture and use*. The implications of this observation include:

- The need for *abstraction mechanisms* that allow the variation of granularity (aggregation abstraction) and sophistication (generalization abstraction) in traceability tasks.
- The need for *inference* services supporting the semantics of the different traceability link types.
- The need for maintaining a baseline of *thick descriptions* of traces (e.g., using multimedia) in case the initially chosen level of trace analysis needs to be reassessed later on.
- The need for mechanisms that allow project managers and developers to define and enact a *model-driven trace process* accompanying the development process.

We discuss these implications in turn. In the final subsection, the design of the KBSA ADM tool by Andersen Consulting illustrates how some of these principles have been applied in practice.

8.1 Abstraction Mechanisms in Trace Representation

The relational representation used in most existing tools does not lend itself naturally to the simultaneous

representation of traces at multiple levels of granularity (e.g., milestone level, document level, fine-grained level). The act of representation entails making judgments about the level of granularity at which the information should be represented. Overly large-grained representations may result in the loss of useful detail, while overly fine-grained representations may create trivial knowledge where the benefits do not warrant the cost of creating such knowledge. Aggregation hierarchies in semantic data models offer an adequate formal means to deal with the granularity problem. However, existing literature does not offer demonstrably effective decision rules for making judgments about granularity; therefore, tools should support multi-granular strategies such as presented in Section 8.4.

A similar problem of overcommitment is faced when considering the generalization abstraction. Such hierarchies allow a smooth transition from low-end to high-end traceability models, at least in the sense that a mix of traces captured with varying sophistication can be interpreted uniformly in terms of the low-end models used.

Finally, the instantiation abstraction allows the definition and change of meta models and, thus, the adaptability of traceability environments to new needs via reorganization and, possibly, the integrated interpretation of traces captured in heterogeneous environments.

In summary, we advocate a move toward more semantics in traceability data models. This move will be facilitated by recent trends in object-relational database technology that offer many of the required extensibility features on top of the proven framework of the relational data model.

8.2 Inference Services

In large projects, the traceability knowledge base can become quite large. As the information needs of participants can vary widely, navigating the entire knowledge base to retrieve relevant information can be very difficult even with sophisticated browsing capabilities. Ability to access relevant information using ad hoc queries can be very helpful.

Inferencing also helps maintain integrity of a knowledge base of interdependent components that get incrementally defined and modified. Such inferencing will be aided by mechanisms for aggregation, classification, and generalization of traceability information.

The ability to make deductive inferences from traceability knowledge base is very helpful in reducing the overhead and increasing the usefulness of the traceability knowledge base. Further, mechanisms to maintain and manage dependencies among various components of requirements traceability can be supported with deductive database capabilities for query processing, deductive rules, and integrity enforcement.

8.3 Grounding Traceability Models in Thick Descriptions

Requirements traceability can be represented in a variety of ways, from *mathematically* formal representations (e.g., transformations that can be used to derive one problem state from another in formal software development) to very

informal representations (e.g., design notebooks that record rationale in natural language) [49].

A primary benefit of formal representations is that they facilitate automated reasoning. However, formal representation of requirements traceability is often difficult as most design situations lack well-defined formal models. Converting informal traceability information into formal models is often extremely labor-intensive and subject to personal bias and choices.

Since design is primarily a collaborative process [57], requirements traceability knowledge often consists of deliberations among individuals engaged in the process. Attempts to represent informal knowledge through formal tools and notations can result in thin descriptions, with the consequence that much of the meaning embedded in such information is lost [58].

Informal representations of requirements traceability can address many of these problems. Such “thick descriptions” [59] enable the user of requirements traceability to grasp the subtleties, tacit and mutual knowledge, and glean descriptions of work practices that are otherwise not made explicit [60]. Finally, unlike formal representations, which can only be used by individuals who are familiar with the rigors of such formalisms, informal representations can be used by a wide set of users. However, informal representations provide another set of problems: The classification, indexing, retrieval, and use of such information in large projects can be impractical. Moreover, though understandable by humans, such information representations are not amenable to automated reasoning. Thus, while informal representations hold much promise with respect to their information content and ease of capture, the difficulty of accessing and reasoning with such information can undermine their utility.

In summary, then, formal and informal representations of requirements traceability complement each other in their respective strengths and weaknesses. Informal representations are easy for capture, whereas formal representations can be manipulated by well-defined inference procedures. Thus, as observed in recent studies on design rationale [61], effective schemes for the capture and use of requirements traceability should seek to combine the advantages of both forms of representations. A tight integration of formal and informal aspects of the requirements traceability information will not only facilitate the capture of various types of information in a format that is most appropriate, but also help use/reuse the captured information in an efficient manner.

8.4 Model-Driven Trace Capture and Usage

Assuming the above-mentioned formal and representational problems have been solved, the project manager or developer can now define the kind of information to be recorded in a trace, i.e., specify the products of traceability. However, the traceability environment still lacks two other important features:

- It provides no means to define the *trace steps* by which the defined information should be recorded. For example, the project manager can define that decisions should be recorded together with their

arguments. But, she cannot define in which situation and by which specific process steps this should happen. In other words, we cannot define the process guidelines according to whether the trace information is captured or used during the development process.

- It provides no tool support for the stakeholders in actually capturing the information according to a defined trace process. Depending on the importance attached to certain trace information in a given situation, the environment could either simply wait for the information to be entered optionally (the default option in most current tools), it could remind the stakeholder to record the information, it could force the stakeholder to do so by blocking the further development process, or it could even record (some of) the information automatically. The development of such an approach requires that the traceability process cannot just be defined, but also effectively embedded in the development process and its supporting CASE environment.

Dömges and Pohl [16] describe a prototypical environment in which selective, model-driven trace capture is possible. Applied to the modeling approach taken in this paper, the approach works roughly as follows: Each basic element in the traceability meta model is seen as a product and associated with process steps that create, delete, or change this product.

In addition to these normal process steps, there are three additional kinds of trace steps. *Supplementary product steps* define how to capture and maintain supplementary information such as design rationale, *process execution steps* record activities in the development environment, and *dependency steps* record dependencies between the artifacts created by process steps, supplementary product steps, and process execution steps. While process execution steps and dependency steps can often be performed largely automatically by simply observing the process in the development environment, the other two kinds of steps usually require human intervention. Based on these conventions, the project manager is thus empowered to define:

- the trace information which should be captured,
- the three kinds of trace steps defining *how* this information should be recorded, and
- the interleaving of the trace steps with the normal process steps, i.e., *when* the trace information is to be captured.

In a sufficiently flexible process-centered software engineering environment, the thus extended process can now be enacted, thus guaranteeing the desired traceability subprocess. However, it must be noted that there are very few process-centered software environments on the market at the moment, so the practical feasibility of this approach can only be assessed from a few case studies conducted with the available prototypes.

8.5 An Example: Multigranular Rationale Capture in KBSA ADM

In this subsection, we use the Rationale Capture part of Andersen Consulting's KBSA ADM tool to illustrate the

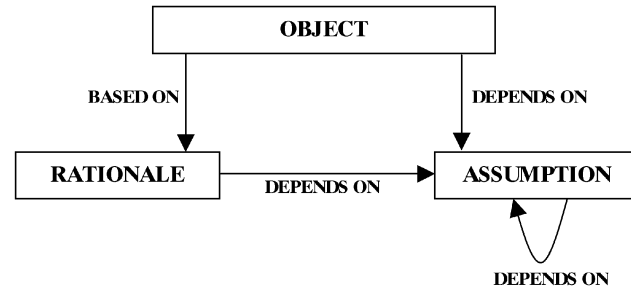


Fig. 19. Simple rationale model.

practical application of the principles discussed in this section: The role of multigranular abstraction, of inference services, formal and informal representations of traceability, and, to some degree, of model-driven trace capture and usage. The combination of these principles is mainly aimed at achieving systematic customizing of traceability needs spanning the distance between the high-end and low-end reference models.

Abstraction in Rationale Models. The design goal of KBSA-ADM was to offer a coherent series of rationale models based on results of the REMAP project [13] for maintaining rationale at different levels of detail.

The model sketched in Fig. 19 is used for capturing rationale at a simple level of detail. It links an OBJECT with its RATIONALE. The model in Fig. 19 also provides for the explicit representation of ASSUMPTIONS and *DEPENDENCIES* among them. Thus, using this model, the assumptions providing justifications to the creation of objects can be explicitly identified and reasoned with. As changes in such assumptions are a primary factor in the evolution of complex systems, this representation will help in the development of support mechanisms to manage such evolution.

In situations where a more detailed representation of rationale is necessary, an intermediate rationale model may be used. In this model, the ISSUES that are *GENERATED* by the OBJECT, as well as DECISIONS that *RESOLVE* them, are explicitly represented. Thus, this intermediate model provides some elementary details of the decision process involved in creating various objects. As before, the ISSUE and DECISION objects can be complex with attributes of their own. Therefore, using this model, a richer context in which objects are created can be captured.

An even finer grained model for representing the detailed deliberations leading to the creation of objects may be more appropriate in critical situations. Empirical studies in the REMAP project [13] suggest that the explicit identification of ALTERNATIVES considered, the ARGUMENTS for and against these ALTERNATIVES, and ASSUMPTIONS behind such ARGUMENTS themselves are part of such a deliberation. A detailed rationale model incorporating all of these primitives is shown in Fig. 20. It incorporates, among other concepts, the Issue Based Information Systems (IBIS) framework [48]. Such models have been used successfully in large scale system development efforts to represent complex decision situations leading to the creation of artifacts.

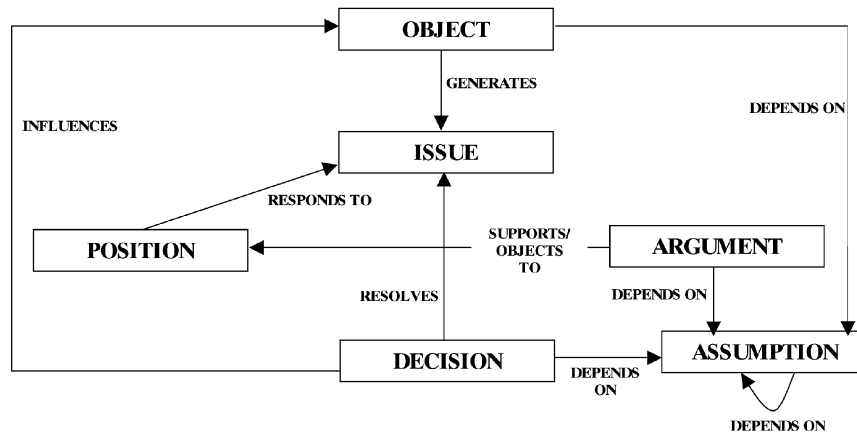


Fig. 20. Detailed rationale model.

The components of this detailed model also closely resemble the Rationale submodel presented in Fig. 4. All the objects used in this model are represented in the Rationale submodel and the major links across these objects are also identified in that model.

The hierarchy of simple, intermediate, and detailed rationale models has been reimplemented in Andersen Consulting's Knowledge Based Software Assistant environment ADM. This implementation has the added advantage that different stakeholders may be interested in different depth of rationale; for example, designers

might use the detailed rationale models, whereas their managers will use the simple rationale view of the same model focusing just on the decisions finally made. Two KBSA-ADM screenshots, using an example from [13], are shown in Figs. 21 and 22.

Inference Services. As the model components have been defined with formal semantics, a system based on these components can support sophisticated reasoning with rationale information. For example, a dependency network of assumptions is managed using a truth maintenance system which propagates the effects of changes in the

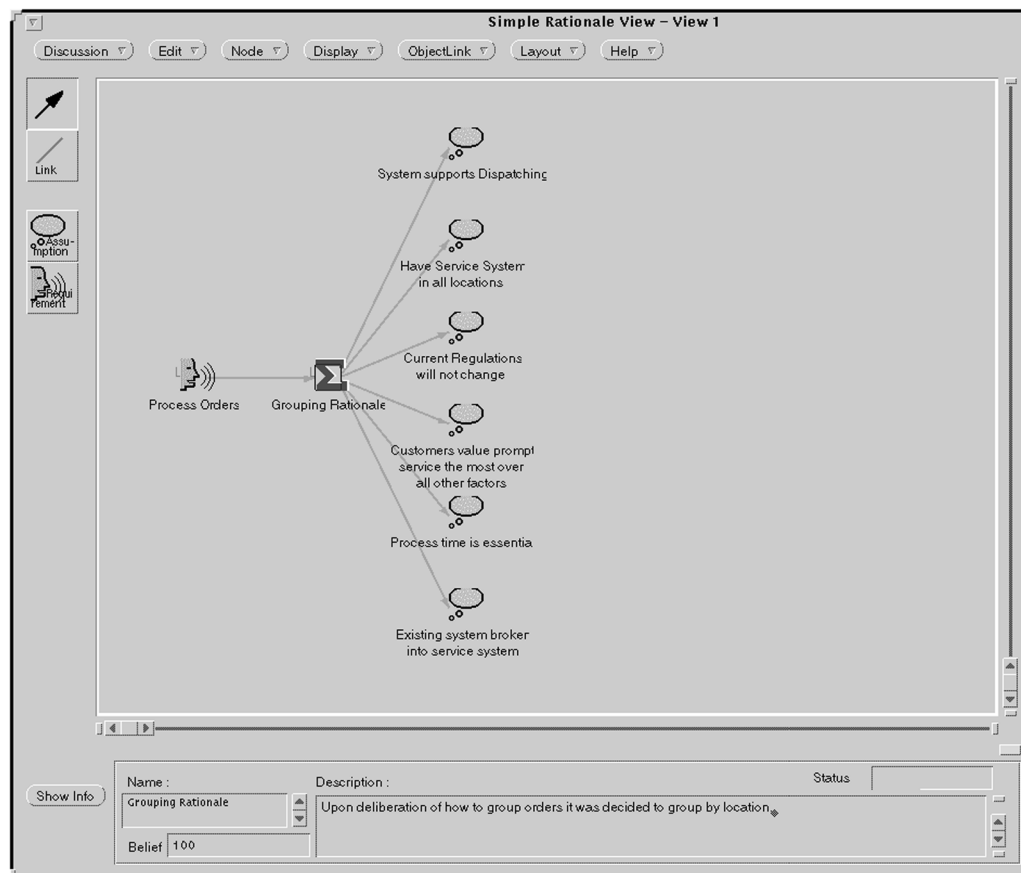


Fig. 21. Rationale in simple view.

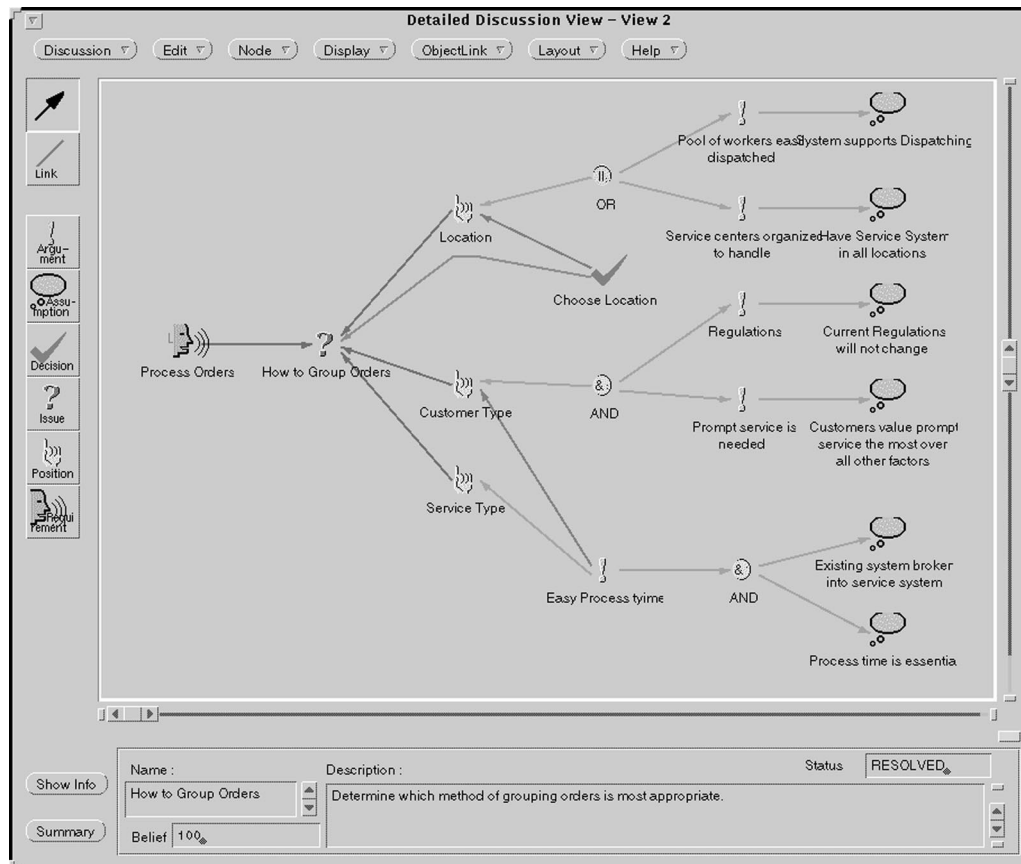


Fig. 22. Rationale in detailed view.

beliefs in assumptions to other components of design rationale knowledge. In the detailed model, such a mechanism will help identify the decisions that need to be reevaluated when changes in assumptions underlying the network of rationale change. KBSA-ADM also supports truth maintenance on such structures, using not just the simple logic-based approach, but also the Dempster-Shafer theory of evidence to determine the belief status of the decisions made.

Thick Descriptions. KBSA-ADM provides a few facilities for integrating formal and informal components of traceability knowledge. First, each object in a discussion can be annotated with multimedia clips. For example, in the discussion involving the processing of orders in a utility company shown in Fig. 22, the node representing location (i.e., graphical location of various services provided by the utility) is annotated with a map of the service area. Further, the tool provides a hypermedia editor within which various objects in our model can be created by simply highlighting a section of a hypermedia document. Such an integration also facilitates easy capture of traceability information.

Model driven capture. KBSA-ADM provides mechanisms for specifying, executing and monitoring process steps. This can be used to define the steps involved a system engineering process and the conditions under which such steps should be executed. Such a facility can be tailored to define the various types of traceability information that should be captured and used. For example, the facility can prompt the user for design rationale information

after major design decisions. An agenda mechanism is used to monitor compliance. However, the current implementation does not include comprehensive model-driven trace capture.

In summary, the KBSA-ADM example illustrates that the advanced tool features discussed in this section are indeed feasible in industrial settings. However, the example also shows that quite a bit of creativity is required to combine these features in a practically useful manner.

9 SUMMARY

We have conducted empirical studies in a broad range of software development organizations to come up with reference models for the objects and traceability links to be recorded. These models are presented at two levels of user sophistication, which could be fairly clearly distinguished in the organizations and mark very different understandings of the importance of requirements traceability.

A simple meta model derived as part of the prestudies also indicates how, in principle, these reference models can be extended by explicit consideration of the different stakeholders involved (contribution and usage structures, cf. [9], [7]). The mapping of the conceptual trace objects to technical media, sources, and documents—the third element in our basic meta model—was addressed by a discussion of how the different traceability link categories can be supported by inference services and what other

requirements on traceability mechanisms the intended usage of the traceability link types implies. The importance of the differentiation between conceptual objects and media is also emphasized in other case studies of RE practice [62].

In summary, we believe that the presented models constitute a good first step toward reference models for requirements traceability. One evidence for this statement is the surprisingly broad acceptance of early versions of several submodels by industry.

The accompanying longitudinal case study indicates that the uptake of these models, if accompanied by appropriate management support and user attitudes, may indeed yield substantial benefits in terms of quality as well as efficiency, leading to improved software process maturity [10]. Of course, this case study will have to be backed up by others which would be greatly helped if our findings concerning the next generation of tools are addressed beyond the present status by vendors. We identified structured knowledge representation, link-type related inference capabilities, the grounding of (possibly multiple) trace models in thick multimedia descriptions, and a model-driven tool-supported trace process as important ingredients. Related work discusses environmental, organizational, and technical factors that influence the implementation of requirements traceability [18]. Our current efforts are devoted to maturing such tool technologies and evaluating them in industrial settings.

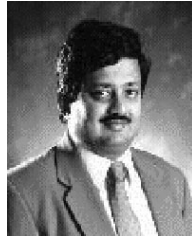
ACKNOWLEDGMENTS

This work was supported in part by grants from the U.S. Office of Naval Research project on Engineering of Complex Systems of the Naval Surface Warfare Center Dahlgren Division, the U.S. Air Force Research Laboratory, the European Commission (ESPRIT project 21.903 CREWS), the Deutsche Forschungsgemeinschaft under project Ja445/5-1 (PRIME), Sonderforschungsbereich 476 IMPROVE, and by a cooperation grant on technical and empirical foundations of requirements traceability jointly funded by the German DAAD and the U.S. National Science Foundation.

REFERENCES

- [1] A.W. Scheer, *Business Process Engineering: Reference Models for Industrial Enterprises*. Springer-Verlag, 1998.
- [2] V. Dhar and M. Jarke, "Learning from Prototypes," *Proc. Sixth Int'l Conf. Information Systems*, pp. 114-133, 1985.
- [3] A. Finkelstein, J. Kramer, and B. Nuseineh, *Software Process Modeling*. London: John Wiley RSP, 1994.
- [4] O. Gotel and A. Finkelstein, "An Analysis of the Requirements Traceability Problem," *Proc. First Int'l Conf. Requirements Eng.*, pp. 94-101, 1994.
- [5] T. Rose, M. Jarke, M. Gocek, C.G. Maltzahn, and H.W. Nissen, "A Decision-Based Configuration Process Environment," *IEEE Software Eng. J.*, vol. 6, no. 5, pp. 333-346, 1991.
- [6] R. Conradi and B. Westfechtel, "Version Models for Software Configuration Management," *ACM Computing Surveys*, vol. 30, pp. 232-282, 1998.
- [7] E. Yu and J. Mylopoulos, "Understanding 'Why' in Software Process Modeling, Analysis and Design," *Proc. 16th Int'l Conf. Software Eng.*, pp. 159-168, 1994.
- [8] O. Gotel and A. Finkelstein, "Contribution Structures," *Proc. Second Int'l Symp. Requirements Eng.*, pp. 100-107, 1995.
- [9] O. Gotel and A. Finkelstein, "Extended Requirements Traceability—Results of an Industrial Case Study," *Proc. Third Int'l Conf. Requirements Engineering*, 1997.
- [10] B. Ramesh, C. Stubbs, T. Powers, and M. Edwards, "Requirements Traceability—Theory and Practice," *Annals of Software Eng.*, vol. 3, pp. 397-415, 1997.
- [11] K. Pohl, *Process Centered Requirements Eng.* Somerset, U.K.: John Wiley Research Studies Press Ltd., 1996.
- [12] V. Dhar and M. Jarke, "Dependency-Directed Reasoning and Learning in Systems Maintenance Support," *IEEE Trans. Software Eng.*, vol. 12, no. 2, pp. 211-227, Feb. 1988.
- [13] B. Ramesh and V. Dhar, "Supporting Systems Development Using Knowledge Captured during Requirements Engineering," *IEEE Trans. Software Eng.*, vol. 18, no. 6, pp. 498-510, June 1992.
- [14] M. Jarke and K. Pohl, "Establishing Visions in Context—Towards a Model of Requirements Processes," *Proc. 14th Int'l Conf. Information Systems*, pp. 23-34, 1993.
- [15] K. Pohl, "The Three Dimensions of Requirements Engineering: A Framework and Its Applications," *Information Systems*, vol. 19, no. 3, pp. 243-258, 1994.
- [16] R. Dominges and K. Pohl, "Adapting Traceability Environments for Project Specific Needs," *Comm. ACM*, vol. 41, pp. 54-63, 1998.
- [17] K. Pohl, K. Weidenhaupt, R. Dönges, P. Haumer, M. Jarke, and R. Klamma, "PRIME—Towards Process—Integrated Modeling Environments," *ACM Trans. Software Eng. and Methodology*, vol. 8, pp. 343-410, 1999.
- [18] B. Ramesh, "Factors Influencing Requirements Traceability Practice," *Comm. ACM*, vol. 41, pp. 37-44, Dec. 1998.
- [19] W.H. Roetzheim, *Developing Software to Government Standards*. New York: Prentice Hall, 1991.
- [20] U.S. Department of Defense, "Military Standard 2167A—Defense System Software Development," Washington, D.C., 1988.
- [21] M. Edwards and S. Howell, "A Methodology for System Requirements Specification and Traceability for Large Real-Time Complex Systems," technical report, U.S. Naval Surface Warfare Center—Dahlgren Division, Dahlgren, Va., 1991.
- [22] J.D. Palmer, "Traceability," *Software Requirements Eng.*, R.H. Thayer and M. Dorfman, eds., pp. 364-374, 1997.
- [23] V.L. Hamilton and M.L. Beeby, "Issues of Traceability in Integrating Tools," *Proc. IEE Colloquium Tools and Techniques for Maintaining Traceability during Design*, Dec. 1991.
- [24] S. Greenspan and C. McGowan, "Structuring Software Development for Reliability," *Microelectronics and Reliability*, vol. 17, 1978.
- [25] B. Ramesh and M. Edwards, "Issues in the Development of a Model of Requirements Traceability," *Proc. Int'l Symp. Requirements Engineering*, pp. 256-259, Jan. 1993.
- [26] G. Stehle, "Requirements Traceability for Real-Time Systems," *Proc. EuroCASE II*, Apr. 1990.
- [27] S. Wright, "Requirements Traceability—What? Why? and How?," *Proc. IEE Colloquium on Tools and Techniques for Maintaining Traceability During Design*, 1991.
- [28] J.D. Fiksel, "New Requirements Management Software Supports Concurrent Eng.," Technical Report: CimFlex Teknowledge Corp., Washington DC, 1994.
- [29] F. Pinheiro and J. Goguen, "An Object-Oriented Tool for Tracing Requirements," *IEEE Software*, pp. 52-65, Mar. 1996.
- [30] M. Dorfman and R.H. Thayer, *Standards, Guidelines, and Examples on System and Software Requirements Engineering*, 1990.
- [31] B.J. Brown, "SEI Curriculum Model," Technical Report SEI-CM-7-1.1, Software Eng. Inst., Pittsburgh, Pa., 1987.
- [32] J.F. Templeton, *Focus Groups: A Guide for Marketing and Advertising Professionals*. New York: Probus Publishing, 1987.
- [33] K.M. Eisenhardt, "Building Theories from Case Study Research," *Academy of Management Review*, vol. 14, pp. 532-550, Oct. 1989.
- [34] D.L. Morgan, *Focus Groups as Qualitative Research*. Newbury Park, Calif.: Sage Publications, 1988.
- [35] M. Jarke, R. Gellersdoerfer, M. Jeusfeld, M. Staudt, and S. Eherer, "ConceptBase—A Deductive Object Base for Meta Data Management," *Int'l J. Intelligent Information Systems*, vol. 5, no. 3, pp. 167-192, 1995.
- [36] J. Mylopoulos, M. Jarke, and M. Koubarakis, "Telos—A Language for Representing Knowledge about Information Systems," *ACM Trans. Information Systems*, vol. 8, pp. 327-362, 1990.
- [37] B. Ramesh, "Representing and Reasoning with Traceability in Model Lifecycle Management," *Annals Operations Research*, vol. 54, pp. 123-145, 1997.

- [38] P.A. Bernstein, T. Bergstraesser, J. Carlson, S. Pal, P. Sanders, and D. Shutt, "Microsoft Repository Version 2 and the Open Information Model," *Information Systems*, vol. 24, pp. 71-98, 1999.
- [39] P. Constantopoulos, M. Jarke, J. Mylopoulos, and Y. Vassiliou, "The Software Information Base—A Server for Reuse," *VLDB J.*, vol. 4, pp. 1-43, 1995.
- [40] M. Jarke, K. Pohl, C. Rolland, and J.R. Schmitt, "Experience-Based Method Evaluation and Improvement: A Process Modeling Approach," *Proc. IFIP 8.1 Working Conf.—CRIS 94*, pp. 1-28, 1994.
- [41] R. Stallman and G. Sussman, "Forward Reasoning and Dependency-Directed Backtracking in a System for Computer-Aided Circuit Analysis," *Artificial Intelligence*, vol. 9, pp. 135-196, 1977.
- [42] P. Winston, *Artificial Intelligence*. Reading, Mass.: Addison-Wesley, 1984.
- [43] E. Charniak and D. McDermott, *Introduction to Artificial Intelligence*. New York: Addison-Wesley, 1985.
- [44] J. Mylopoulos, L. Chung, and B. Nixon, "Representing and Using Nonfunctional Requirements: A Process-Oriented Approach," *IEEE Trans. Software Eng.*, vol. 18, no. 6, pp. 483-497, June 1992.
- [45] H. Simon, *The Sciences of the Artificial*. Cambridge, Mass.: MIT Press, 1981.
- [46] J.R. Hauser and D. Clausing, "The House of Quality," *Harvard Business Review*, pp. 63-73, May/June 1988.
- [47] H. Hau and R. Kashyap, "Belief Combination and Propagation in a Lattice-Structured Network," *IEEE Trans. Systems, Man, and Cybernetics*, vol. 20, pp. 45-58, 1990.
- [48] J. Conklin and M.L. Begeman, "gIBIS: A Hypertext Tool for Exploratory Policy Discussion," *ACM Trans. Office Information Systems*, vol. 6, pp. 303-331, Oct. 1988.
- [49] J. Lee, "Design Rationale Capture and Use," *AI Magazine*, vol. 14, pp. 24-26, 1993.
- [50] R. Goldman-Segall, "Collaborative Virtual Communities Using Learning Constellations: A Multimedia Research Tool," *Socio-media: Multimedia, Hypermedia and the Social Construction of Knowledge*, E. Barrett, ed., Cambridge, Mass: MIT Press, 1992.
- [51] J. Rochelle, R. Pea, and R. Trigg, "Videonoter: A Tool for Exploratory Video Analysis," Technical Report IRL-90-0021, Institute for Research on Learning, 1990.
- [52] F. Lakin, J. Wambaugh, L. Leifer, D. Cannon, and C. Sivard, "The Electronic Design Notebook: Performing Medium and Processing Medium," *Int'l J. Computer Graphics*, vol. 5, pp. 214-226, 1989.
- [53] Ascent Logic Corporation, *RDD-100: Requirements Driven Design, User Guide*. June 1992.
- [54] QSS Ltd., *Dynamic Object Oriented Requirements System—Reference Manual*. Oxford, UK, 1996.
- [55] TD Technologies Inc., *SLATE User Manual*. 1996.
- [56] Cadre Technologies, *Cadre/RQT User Manual*. 1992.
- [57] G. Fischer, J. Grudin, A. Lemke, R. McCall, J. Ostwald, B. Reeves, and F. Shipman, "Supporting Indirect Collaborative Design with Integrated Knowledge-Based Design Environments," *Human-Computer Interaction J.*, vol. 7, no. 3, pp. 281-314, 1992.
- [58] R.C. Anderson, C. Heath, P. Luff, and T. Moran, "The Social and the Cognitive in Human-Computer Interaction," Technical Report EPC-91-126, Rank Xerox Euro PARC, Cambridge, UK, 1991.
- [59] C. Geertz, *Thick Description: Toward an Interpretive Theory of Culture*. New York: Basic Books, 1973.
- [60] B. Jordon, "Technology and Social Interaction: Notes on the Achievement of Authoritative Knowledge in Complex Settings," Technical Report IRL-92-0027, Inst. for Research on Learning, Palo Alto, Calif., 1992.
- [61] S. Shum and N. Hammond, "Argumentation-Based Design Rationale: What Use and What Cost?" *Int'l J. Human Computer Studies*, vol. 40, pp. 603-652, 1994.
- [62] H.W. Nissen, M.A. Jeusfeld, M. Jarke, G. Zemanek, and H. Huber, "Managing Requirements Perspectives with Meta Models," *IEEE Software*, pp. 47-58, Mar. 1996.



Balasubramaniam Ramesh received his PhD in information systems from New York University. He is an associate professor and director of Research and Sponsored Programs in the Department of Computer Information Systems at Georgia State University. His research work has appeared in several leading conferences and journals including the *IEEE Transactions on Software Engineering*, *IEEE Expert*, *Annals of Software Engineering*, *Annals of Operations Research*, *Concurrent Engineering—Research and Applications*, *Decision Support Systems*, and *Journal of Systems Integration*. His research interests include supporting collaborative work with artificial intelligence, decision support, and multimedia technologies in the areas of requirements engineering and traceability in systems development, concurrent engineering, new product development, knowledge management, and business process redesign. His other research interests include data mining and e-services. He is a member of the IEEE.



Matthias Jarke received diploma degrees in business administration and computer science, as well as a doctorate from the University of Hamburg, Germany. He is a professor of information systems and chair of the Computer Science Department at Aachen University of Technology (RWTH Aachen). He also served on the faculties of New York University and the University of Passau prior to joining Aachen in 1991. His research interests span all aspects of information systems support for cooperative engineering processes. In this domain, he has been coordinator of three European Esprit projects (DAIDA, NATURE, and CREWS), and participated in many others. He is the European editor of the *Journal, Information Systems*, and has served as program chair of several International Conferences, including VLDB, EDBT, CoopIS, CaiSE, and the German National Computer Science Conference, Informatik 1997. In 1999, he was elected vice president of the German Computer Society, and appointed executive director of GMD-FIT, Institute of Applied Technology within the National IT Labs in Birlinghover, Germany. He is a senior member of the IEEE and has served as an associate editor of *IEEE Transactions on Software Engineering* from 1988-1992.